# REFACTORING

**Es el proceso a través del cual se cambia un sistema de software para mejorar la organización, legibilidad, adaptabilidad y mantenibilidad del código luego que ha sido escrito** → Transformación de código que preserva el comportamiento pero mejora el diseño.

## Definiciones clave

- **Refactoring (sustantivo)**:Es el cambio hecho a la estructura interna del software para hacerlo más fácil de entender y más barato de modificar sin cambiar su comportamiento observable.
    - Cada refactoring tiene un nombre específico y una secuencia de pasos ordenados "mechanics".
    - Los cambios están catalogados.
- **Refactor (verbo):** Reestructurar el software aplicando una serie de refactorings sin cambiar su comportamiento observable.

## Mecánica del refactoring

Se realiza a través de un conjunto de pasos atómicos para transformar el código.

## Usos más comunes del refactoring

- Eliminar duplicaciones
- Simplificar lógicas complejas
- Clarificar códigos

## Escenarios para aplicar refactoring

- Una vez que tengo código que funciona y pasa los tests
- A medida que voy desarrollando:
    - cuando encuentro codigo difícil de entender(ugly code)
    - cuando tengo que hacer un cambio y necesito reorganizar primero
- **Importante:** Siempre testear después de cada cambio.

## ¿Cuándo hay que refactorizar?

- Al agregar funcionalidad:
    - Tengo que entender el código para modificarlo/agregarle nueva funcionalidad, entonces refactorizar para clarificar el código y así entenderlo más fácilmente
    - Agregar funcionalidad más fácilmente

- Cuando se necesita arreglar un bug:
    - Un bug puede ser una señal de que el código necesita refactorización, ya que no era lo suficientemente claro para darse cuenta de que había un bug.
    - Durante revisiones de código.

## Consideraciones importantes!!!!!

➔ Refactoring **no agrega funcionalidad**; puede complicar los deadlines si se intenta hacer ambas cosas simultáneamente.

➔ La mejor manera de optimizar un programa es primero escribirlo bien factorizado y luego optimizarlo.

### Leyes de Lehman relacionadas

- Cambio Continuo:
    - Los sistemas deben adaptarse continuamente o se vuelven progresivamente menos satisfactorios.
- Crecimiento Continuo:
    - La funcionalidad de un sistema debe ser incrementada continuamente para mantener la satisfacción del cliente.
- Complejidad Creciente:
    - A medida que un sistema evoluciona su complejidad se incrementa a menos que se trabaje para evitarlo.
- Calidad Decreciente
    - la calidad decae sin mantenimiento riguroso.

## The two hats-"Los dos sombreros" del desarrollo con refactoring

Cuando se usa refactoring para desarrollar software, se divide el tiempo entre dos actividades distintivas:

- Agregar funcionalidad:
  - No se cambia el código existente.
  - El progreso se mide agregando test y probandolos
- Refactoring
  - Solo se reestructura el código, sin agregar funcionalidad.
  - No se agregan test (a menos que se encuentre un caso perdido anteriormente). Solo se cambian tests cuando realmente se necesita para hacer frente a un cambio en la interfaz.

Mientras se desarrolla software, estos sombreros se cambian frecuentemente pero durante este tiempo se debe ser consciente del sombrero que se está usando → no se pueden usar los en simultáneo.

## Ventajas de un buen diseño (refactorizado)

Un diseño pobre generalmente toma más código para hacer las mismas cosas, muchas veces porque el código hace básicamente lo mismo en muchos lugares.Reducir la cantidad de código no hace que el programa corra más rápido pero:

- Reduce código repetido (el código debe decir cada cosa solo una vez) → Esencia de un buen diseño.

No hace que el programa corra más rápido, pero mejora la mantenibilidad y claridad.

# Code Smell:

**Indicadores de posibles problemas en el diseño del código fuente.** Destacan áreas en las que el código podría estar violando principios de diseño fundamentales

- No son errores, pero hace que futuros desarrollos puedan:
  -Ser más difíciles
  -Demandar más horas hombre ⇒costo

Conceptos relacionados:

- **Velocity:**

  Cantidad de tareas completadas en un periodo de tiempo:

  (funcionalidad)/periodo de tiempo

- **Deuda técnica:**

  Costo de rehacer una tarea que tiempo atrás fue resuelta con una solución rápida y desprolija(quick and dirty)

  Concepto que introdujo Ward Cunningham para explicar a los stakeholders la necesidad de refactoring

  - Está bien tomar prestado o endeudarse cuando estratégicamente conviene entregar código rápido para ganar feedback y aprender, y el código refleja nuestro entendimiento actual del problema
  - El peligro es cuando esa deuda no se paga
  - Cada minuto dedicado a código que acarrea deuda cuenta como interés. Y hasta que no se aplique refactoring para pagar esa deuda, seguiremos acumulando interés
  - El código que escribimos debería ser lo suficientemente limpio para refactorizarlo fácilmente a medida que lo entendemos mejor
  - Capital de la deuda: costo de remediar los problemas de diseño (costo del refactoring)
  - Interés de la deuda: costo adicional o esfuerzo extra por acarrear un mal diseño

  Varias IDEs tienen la capacidad de cuantificar y visualizar el capital de la TD

# Refactoring y CLEAN code:

Refactoring permite crear código limpio (CLEAN) y mantenible.

**CLEAN:**

- ➔ **C**ohesive → cohesivo
- ➔ **L**oosely coupled → poco acoplado
- ➔ **E**ncapsulated → encapsulado
- ➔ **A**ssertive → claro y explícito
- ➔ **N**on-redundant → sin redundancia

Pero además: **legible**

## ¿Por qué el refactoring es importante?

- Facilita la comprensión del código
- Reducir el costo de mantenimiento debido a los cambios inevitables que sufrirá el sistema (por ejemplo, código duplicado que haya que cambiar)
- Facilitar la detección de bugs
- Permite agregar funcionalidad más rápido en el futuro. (super importante!!)

## ¿Cuándo aplicar refactoring?

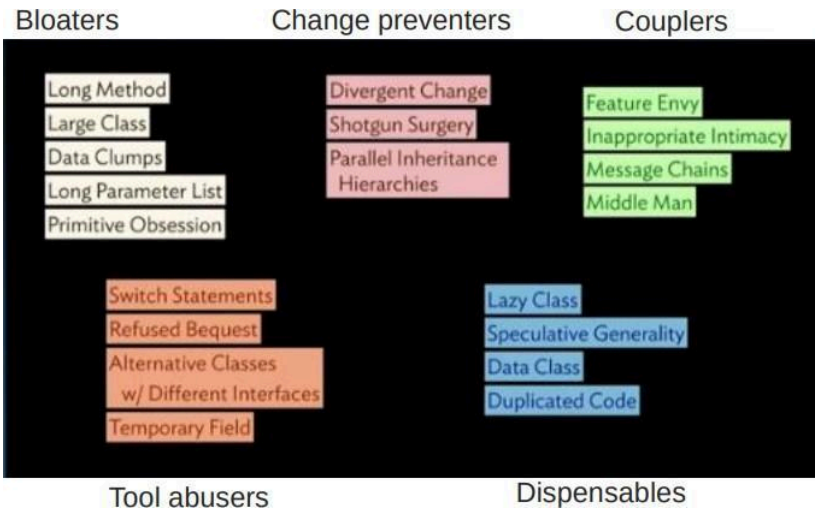
- ➔ En el contexto de TDD
- ➔ Cuando se descubre código con mal olor, aprovechando la oportunidad dejarlo al menos un poco mejor, dependiendo del tiempo que lleve y de lo que esté haciendo
- ➔ Cuando no se entiende el código y se logra entenderlo
- ➔ Cuando encuentro una mejor manera de codificar algo

## Automatización del refactoring

Existen herramientas que facilitan y hacen menos costoso el proceso:

- Son potentes para realizar refactorings útiles.
- Restringen cambios para preservar el comportamiento (uso de precondiciones) → Restrictivas
- Son interactivas para verificar condiciones rápidamente. →Interactivas
- Solo chequean lo que sea posible desde el árbol de sintaxis y la tabla de símbolos
- Pueden ser conservadoras, evitando refactorings si no pueden garantizar seguridad, preservación de comportamiento o asumir buenas técnicas de programación→Conservativas

# Categorización de Code Smells




**Bad Smells - Code Smells**
Indicadores o señales de problemas en el código que sugieren que es necesario aplicar refactorizaciones para mejorar diseño, legibilidad y mantenimiento.

## BLOATERS (Código inflado/ hinchado)

### Large class:

- Descripción:
  - Una clase intenta hacer demasiado trabajo
  - Tiene muchas variables de instancia
  - Tiene muchos métodos
- Problemas
  - Indica un problema de diseño (baja cohesión).
  - Algunos métodos pueden pertenecer a otra clase.
  - Generalmente tiene código duplicado.
- Refactoring a usar:
  - Extract Class.
  - Extract Subclass.

### Long method:

- Descripción:

    - Un método tiene muchas líneas de código (como límite 20 pero depende del lenguaje).
- Problemas
    - Cuanto más largo es un método, más difícil es entenderlo, cambiarlo y reusarlo.
- Refactoring a usar:
    - Extract Method.
    - Decompose Conditional.
    - Replace Temp with Query.

**Long parameter list:**

- Descripción:
    - Un método con una larga lista de parámetros.
- Problemas:
    - Difícil de entender y usar.
    - Dificulta la reutilización (es más difícil obtener todos los parámetros para pasarlos en la llamada):
    - NOTA: La excepción es cuando no quiero crear una dependencia entre el objetos llamador y el llamado
- Refactoring a usar:
    - Replace Parameter with Method
    - Preserve Whole Object
    - Introduce Parameter Object.

### Data Clumps (Agrupamientos de datos)

- Descripción:
  - Variables que siempre aparecen juntas en muchas partes del código y deberían estar en una clase propia.
- Problemas:
  - Indica una entidad conceptual no representada como clase.
- Refactoring a usar:
  - Extract class (si aparecen como campos)
  - Introduce Parameter Object (para parámetros)
  - Preserve Whole Object (si vienen de un mismo objeto)

### Primitive Obsession (Obsesión por primitivos)

- Descripción:
  - Uso excesivo de tipos primitivos en lugar de objetos con significado.
- Problemas:
  - Disminuye la expresividad del código
  - Dificulta validaciones
  - Refuerza estructuras rígidas
- Refactoring a usar:
  - Replace Data Value with Object (Reemplazar valor por objeto).
  - Replace Type Code with Class/Subclasses/State-Strategy.
  - Extract Class.
  - Introduce Parameter Object.
  - Replace Array with Object

## CHANGE PREVENTERS (Impedidores de cambio):

### **Divergent change** (cambio divergente)**:**

- Descripción:
  - Una clase cambia por muchas razones distintas.
- Problemas:
  - Indica que una clase tiene múltiples responsabilidades (violación de SRP, principios solid)

    - Complica el mantenimiento y las pruebas
- Refactoring a usar:
    - Extract class

**Shotgun surgery** (Cirugía de escopeta)**:**

- Descripción:
    - Un cambio requiere modificar muchas clases distintas con pequeñas modificaciones dispersas.
- Problemas:
    - Alta dispersión del comportamiento relacionado
    - Aumenta el riesgo de errores.
    - Dificulta la trazabilidad del cambio
    - Rompe el principio de cohesión
- Refactoring a usar:
    - Move method
    - Move field

**Parallel Inheritance hierarchies** (Jerarquías paralelas)**:**

- Descripción:
    - Crear una subclase en una jerarquía implica crear una en otra jerarquía paralela.
- Problemas:
    - Jerarquías acopladas.
    - Diseño rígido y propenso a errores al extender el sistema.
- Refactoring a usar:
    - Move method
    - Move field

## COUPLERS (Acopladores):

**Feature envy:**

- Descripción:

    - Un método en una clase usa principalmente los datos y métodos de otra clase para realizar su trabajo (se muestra “envidiosa” de las capacidades de otra clase).
- Problemas:
    - Indica un problema de diseño (generalmente se prefiere que los datos y las acciones sobre los datos estén en la misma clase → el método está mal ubicado)
    - Diseño rígido y propenso a errores al extender el sistema.
- Refactoring a usar:
    - Move methodre
    - Extract method

### Inappropriate intimacy:

- Descripción:
    - Dos clases acceden a detalles internos (atributos o métodos) una de la otra (están muy acopladas).
- Problemas:
    - Alto nivel de dependencia que rompe la encapsulación
    - Los cambios en una clase afectan directamente a la otra.
    - Dificulta el mantenimiento y la evolución del software.
- Refactoring a usar:
    - Move method and move field
    - Change Bidirectional Association to Unidirectional
    - Extract Class
    - Hide Delegate
    - Replace
    - Delegation with Inheritance.

### **Middle man** (Intermediario)

- Descripción:
    - Una clase delega demasiado trabajo a otras, sin aportar lógica propia.
- Problemas:
    - Intermediario inútil que solo redirige mensajes.
- Refactoring a usar:
    - Remove middle man.

### Message chains

- Descripción:
  - El cliente accede a un objeto a través de una larga cadena de llamadas: a.getB().getC().getD()
- Problemas:
  - El cliente queda muy acoplado a la estructura interna del sistema.
- Refactoring a usar:
  - Hide Delegate
  - Move Method
  - Extract Method

## TOOL ABUSERS

### Switch statements

- Descripción:
  - Cuando sentencias condicionales contienen lógica para diferentes tipos de objetos
  - Cuando todos los objetos son instancias de la misma clase, eso indica que se necesitan crear subclases.
- Problemas:
  - La misma estructura condicional aparece en muchos lugares → Código duplicado
  - Dificulta el mantenimiento
- Refactoring a usar:
  - Move Method
  - Extract Method
  - Replace type code with subclasses or Replace type code with state/strategy
  - Replace conditional logic with polymorphism
  - Replace parameters with explicit methods
  - Introduce null object

### Alternative classes with different interfaces

- Descripción:
  - Clases similares que hacen cosas parecidas pero con interfaces distintas.

- Problemas:
    - Dificulta la reutilización y la intercambiabilidad.
- Refactoring a usar:
    - Rename Method
    - Move Method
    - Extract Superclass o Extract Interface

### Refused bequest

- Descripción:
    - Una subclase hereda métodos o propiedades que no necesita o no usa.
- Problemas:
    - La herencia se fuerza, generando confusión y acoplamiento innecesario.
- Refactoring a usar:
    - Push down method/field
    - Replace inheritance with delegation

### Temporary field

- Descripción:
    - Atributos que solo son usados en ciertos casos o métodos específicos.
- Problemas:
    - Dificultan entender el estado completo del objeto.
- Refactoring a usar:
    - Extract class.
    - Introduce null object.

## DISPENSABLES (Prescindibles)

### Duplicated code

- Descripción:
    - El mismo código, o código muy similar, aparece en muchos lugares.
- Problemas:
    - Código más largo de lo que debería ser
    - Difícil de cambiar
    - Difícil de mantener

    - Un bug fix en un clone no es fácilmente propagado a los demás clones
- Refactoring a usar:
    - Extract Method
    - Pull Up Method
    - Form Template Method
    - Substitute Algorithm

**Data class**

- Descripción:
    - Una clase que solo tiene variables y getters/setters para esas variables
    - La clase actúa unicamente como contenedor de datos
- Problemas:
    - Generalmente sucede que otras clases tienen métodos con "envidia de atributos"
    - Suele indicar que el diseño es procedural.
    - Difícil de cambiar
    - Difícil de mantener
    - Un bug fix en un clone no es fácilmente propagado a los demás clones
- Refactoring a usar:
    - Move method

**Lazy class**

- Descripción:
    - Una clase que no hace lo suficiente como para justificar su existencia.
    - Puede haber sido útil antes, pero ahora contiene muy poca funcionalidad.
- Problemas:
    - Complejidad innecesaria
    - Gasto sin beneficio.
- Refactoring a usar:
    - Collapse hierarchy
    - Inline class

**Speculative Generality**

- Descripción:
    - Código que fue escrito para un uso futuro que nunca llegó: clases base vacías, métodos no utilizados, abstracciones

innecesarias.

- Problemas:
  - Dificulta entender y mantener el diseño..
- Refactoring a usar:
  - Collapse hierarchy
  - Inline class
  - Remove parameter
  - Remove method

**Comments (Comentarios excesivos)**

- Descripción:
  - Se refiere al uso excesivo o innecesario de comentarios para explicar código que podría ser claro por sí mismo.
- Problemas:
  - Síntoma de código confuso.
- Refactoring a usar:
  - Extract Method
  - Rename Method

# Catálogo de refactorings (Fowler)

## Composición de métodos

- Permiten "distribuir" el código adecuadamente.
- Métodos largos son problemáticos.Contienen:
  - mucha información
  - lógica compleja

**Extract Method**

- Motivación :
  - Métodos largos
  - Métodos muy comentados
  - Incrementar reuso
  - Incrementar legibilidad
- Mecánica:
  1. Crear un nuevo método cuyo nombre explique su propósito

2. Copiar el código a extraer al nuevo método
3. Revisar las variables locales del original
4. Si alguna se usa sólo en el código extraído, mover su declaración
5. Revisar si alguna variable local es modificada por el código extraído. Si es solo una, tratar como query y asignar. Si hay más de una no se puede extraer.
6. Pasar como parámetro las variables que el método nuevo lee.
7. Compilar
8. Reemplazar código en método original por llamada
9. Compilar y testear

**Inline Method**

- Motivación:
  - El cuerpo de un método es tan claro como su nombre.
  - Cuando tenés un conjunto de métodos que parecen estar mal fragmentados, podés incluirlos todos dentro un único método grande y luego volver a extraer métodos de forma mas coherente.
- Mecánica:
  - Verificar que el método no sea polimórfico.
    - No hacer inline si las subclases lo sobrescriben; no pueden sobrescribir un método que ya no existe.
  - Buscar todas la invocaciones al método .
  - Reemplazar cada llamada con el cuerpo del método.
  - Compilar y testear
  - Eliminar la definición del método..

**Replace Temp with Query**:

- Motivación: usar este refactoring:
  - Para evitar métodos largos. Las temporales, al ser locales, fomentan métodos largos

- Para poder usar una expresión desde otros métodos
- Antes de un Extract Method, para evitar parámetros innecesarios

Solución:

- Extraer la expresión en un método
- Remplazar TODAS las referencias a la var. temporal por la expresión
- El nuevo método luego puede ser usado en otros métodos

- Mecánica:
    1. Encontrar las vars. temporales con una sola asignación (si no, Split Temporary Variable)
    2. Extraer el lado derecho de la asignación (tener cuidado con los efectos colaterales; si no, Separate Query From Modifier)
    3. Remplazar todas las referencias de la var. temporal por el nuevo método
    4. Eliminar la declaración de la var. temporal y las asignaciones
    5. Compilar y testear

**Split Temporary Variable**

- Motivación:
    - Si una misma variable se usa para varias cosas diferentes (por ejemplo, para guardar resultados intermedios distintos), puede ser confuso o peligroso.
- Mecánica:
    - Cambiar el nombre de una variable temporal en su declaración y en su primera asignación.
    - Declarar la nueva variable temporal como **final**
    - Cambiar todas las referencias a la variable hasta su segunda asignación.
    - Declarar una nueva variable temporal en el

punto de la segunda asignación.
    - Compila y testear.
    - Repetir el proceso en etapas, en cada etapa cambiando el nombre en la declaración y modificando las referencias hasta la siguiente asignación.

**Replace Method with Method Object**

- Motivación:
    - Tenés un método muy largo que usa variables locales de una manera que impide aplicar Extract Method (no podés extraer partes porque las variables locales generan dependencia difíciles de separar)

- Mecánica:
    - Crear una nueva clase y nombrarla igual que el método.
    - Agregar a la nueva clase
        - Un campo final para el objeto que contenía el método original (el objeto fuente)
        - Un campo para cada variable temporal y cada parámetro del método.
    - Agregar a la nueva clase un constructor que reciba el objeto fuente y cada parámetro.
    - Agregar a la nueva clase un método llamado compute.
    - Copiar el cuerpo del método original dentro de compute.
        - Usar el campo del objeto fuente para cualquier invocación de métodos sobre el objeto original.
    - Compilar.
    - Reemplazar el método antiguo por uno que cree una instancia del nuevo objeto y llame a compute.

### Substitute Algorithm

- Motivación:
  - Cuando un algoritmo está implementado de forma complicada o ineficiente o puede ser reemplazado por otro más claro o rápido
  - Cuando el algoritmo es difícil de entender o hay otra forma más directa de hacerlo
- Mecánica
  - Identificar el algoritmo actual dentro del método
  - Escribir una nueva implementación más simple o eficiente
  - Reemplazar el código viejo por el nuevo
  - Ejecutar pruebas para validar que el comportamiento no cambió

## Mover aspectos entre objetos:

- Ayudan a mejorar la asignación de responsabilidades.

### Move Method

- Motivación**:**
  - Un método está usando o usará muchos servicios que están definidos en una clase diferente a la suya (Feature envy)
  - Solución**:**
    - Mover el método a la clase que contiene los servicios que usa.
    - Convertir el método original en un simple delegación o eliminarlo
- Mecánica**:**
  1. Revisar las variables de instancia que utiliza el método. ¿Tiene sentido moverlas también?
  2. Chequear subclases y superclases de la clase original por si hay otras declaraciones del método (puede que no se pueda mover)

3. Declarar el método en la clase destino
4. Copiar y ajustar el código (ajustando las referencias desde el objeto origen al destino); chequear manejo de excepciones
5. Convertir el método original en una delegación
6. Compilar y testear
7. Decidir si eliminar el método original → eliminar las referencias
8. Compilar y testear

**Move Field**

- Motivación**:**
    - Un campo (atributo) es utilizado más por otra clase que por la clase donde está definido originalmente, es señal de que dicho campo debería moverse.
    - Solución**:**
        - Crear un campo en la otra clase y redirigir todos los usos del campo antiguo a ese nuevo campo
- Mecánica**:**
    1. Si el campo es público, primero aplicar Encapsulate Field
        a. Si muchos métoso acceden al campo, puede ser putil aplicar Self Encapsulate Field antes.
    2. Compilar y testear
    3. Crear un campo equivalente en la clase destino, junto con sus métodos getter y en caso de ser necesario, setter
        a. Compilar la clase destino
    4. Determinar cómo obtener una referencia la objeto destino desde la clase original
        a. Puede existir ya un método o campo que devuelva el objeto destino.
        b. Si no existe, crear un método que lo proporcione.
        c. En la última instancia, puede ser necesario crear un nuevo campo temporal en la clase de origen

para almacenar la referencia.

5. Eliminar el campo de la clase original
    a. Acceso  llamar al getter del objeto destino.
    b. Asignación  llamar al setter del objeto destino.
6. Reemplazar todas la referencias la campo antiguo por llamadas al getter/setter del campo en la clase destino
7. Si el campo no era privado, revisar también las subclases por posibles accesos directos
8. Compilar y testear después de cada paso.

**Extract class**:

- Motivación**:**
    - Una clase maneja demasiadas cosas distintas:
        - Resulta difícil de entender.
        - Tiende a tener muchos métodos y muchos campos
        - Aparecen grupos de atributos y métodos que siempre cambian juntos
        - Existen partes que podrían evolucionar de forma separada.
    - Se pueden identificar subconjuntos de datos y métodos que claramente perteneces a otra abstracción.
    - Solución**:**
        - Crear una nueva clase y colocar allí los campos y métodos responsables de la funcionalidad relevante
- Mecánica**:**
    1. Decidir cómo dividir las responsabilidades dentro de la clase original
    2. Crear una nueva clase para representar las responsabilidades separadas.
        - Si la clase original deja de tener un nombre adecuado, renombrarla.
    3. Crear un enlace desde la clase original hacia la nueva.
        - Podría necesitarse un enlace bidireccional, pero solo

agregarlo cuando sea realmente necesario.

4. Mover los campos apropiados a la nueva clase usando Move Field.
    - Compilar y testear después de cada movimiento.
5. Mover los métodos que correspondan usando Move Method.
    - Comenzar con los métodos de bajo nivel (los que son llamados por otros métodos )
    - Avanzar luego hacia los métodos de mayor nivel
    - Compilar y testear después de cada método movido.
6. Revisar y simplificar la interfaz de ambas clases.
    - SI se uso un enlace bidireccional, revisar si puede reducirse a un enlace unidireccional.
7. Decidir si la nueva clase será expuesta hacia el exterior

    - Si se expone, determinar si debe ser un objeto por referencia o un objeto de valor inmutable

**Inline class**

- Motivación**:**
    - Una clase no hace casi nada, no es responsable de nada. Ya no está aportando suficiente valor y no justifica su existencia
    - Ocurre muy frecuentemente después de aplicar otros refactors
    - Solución**:**
        - Eliminarla y fusionar su contenido en otra clase que la utilice más
- Mecánica**:**
    1. Declarar el protocolo público de la clase origen en la clase destino
        a. Inicialmente hacer que estos métodos deleguen al objeto de la clase origen
        b. Si conviene tener una interfaz separada para esos métodos, usar Extract interface
    2. Cambiar todas las referencias a la clase origen para que apunten a la clase absorbente
        a. Hacer la clase origen private para evitar usos

externos

b. Renonmbrar la clase origen para que el compilador detecte referencias sin actualizar

3. Compilar y testear
4. Mover métodos y atributos desde la clase origen hacia la clase absorbente, usando Move Method y Move Field
    a. Repetir hasta que no quede nada útil en la clase origen
5. Eliminar la clase origen

**Remove middle man**

- Motivación**:**
    - Una clase tiene muchos métodos que solamente delegan a otros objetos.
    - Solución**:**
        - Borrar estos métodos y forzar al cliente a llamar a los métodos finales directamente
- Mecánica**:**
    1. Crear un accessor para el delegado (si aún no existe).
    2. Para cada uso que hace el cliente de un método del delegado:
        a. Eliminar el método delegador del servidor
        b. Modificar al cliente para que invoque directamente al delegado.
    3. Compilar y testear después de cada cambio

**Hide delegate**

- Motivación**:**
    - Cuando un cliente llama a un método definido en uno de los campos del servidor. Ejemplo; servidor.getDelegado().hacerAlgo() □ el cliente está obligado a conocer al objeto delegado y si el

delegado cambia, el cliente probablemente también deberá cambiar

- Solución**:**
  - Agregar un método delegador en el servidor, que oculte al delegado.Así los cambios se limitan al servidor y no se propagan al cliente.
  - Convertir el método original en un simple delegación o eliminarlo

- Mecánica**:**
  1. Por cada método del delado, crear un método delegador equivalente en el servidor
  2. Ajustar el cliente para que llame al servidor, en vez de llamar directamente al delegado
  3. Compilar y testear
  4. Si ya ninguna cliente necesita acceder al delegado, eliminar el accessor (por ejemplo getDelegado()) del servidor.
  5. Compilar y testear de nuevo.

## Organización de datos

- Facilitan la organización de atributos

### Self Encapsulate Field

- Motivación: Evita el acceso directo a atributos. Facilita futuras modificaciones o validaciones sin cambiar el código cliente.
- Mecánica:
  1. Crear getters y setters para el campo.
  2. Reemplazar el acceso directo al campo por los métodos.
  3. Cambiar la visibilidad del campo a private.

### Encapsulate Field / Collection

- Motivación: Protege la representación interna. Previene

cambios inesperados desde afuera.

- Mecánica:
    1. Crear métodos getX() y setX() para campos simples.
    2. Para colecciones, evitar exponer la colección directamente.
        - Usar getItems() que devuelva copia o unmodifiableList.
        - Agregar addItem() / removeItem() en vez de modificar directamente.

### Replace Data Value with Object

- Motivación: Cuando un dato simple (como un string) empieza a tener comportamiento o reglas propias.
- Mecánica:
    1. Crear una nueva clase para representar el dato.
    2. Reemplazar el campo por una instancia de esa clase.
    3. Mover cualquier lógica relacionada al nuevo objeto.

### Replace Array with Object

- Motivación: Cuando una array guarda distintos tipos de datos relacionados, pero sin nombres claros ni estructura.
- Mecánica:
    1. Crear una clase que encapsule la estructura.
    2. Reemplazar el uso del array por una instancia de la nueva clase con atributos con nombre.

### Replace Magic Number with Symbolic Constant

- Motivación: Mejora la legibilidad. Permite saber el significado de los números usados.
- Mecánica:
    1. Declarar una constante con un nombre significativo.
    2. Reemplazar todas las apariciones del número por la constante.

## Simplificación de expresiones condicionales

- Ayudan a simplificar los condicionales

### Decompose Conditional

- Motivación: Divide condicionales complejas en partes legibles.
- Mecánica:
    1. Extraer cada condición y cada rama (then, else) a métodos separados con nombres descriptivos.

### Consolidate Conditional Expression

- Motivación: Cuando hay varias condiciones que hacen lo mismo, combinarlas para evitar duplicación.
- Mecánica:
    1. Unir las expresiones con operadores lógicos (&&, ||).
    2. De ser necesario, extraer la condición consolidada a un método.

### Consolidate Duplicate Conditional Fragments

- Motivación:Identical code can be found in all branches of a conditional.
- Mecánica:
    1. Mover el código duplicado fuera de la estructura condicional.

### Replace Conditional with Polymorphism:

- Motivación: Cuando hay muchas estructuras condicionales (if/switch) basadas en el tipo de objeto.
- Mecánica:

    1. Crear la jerarquía.

    2. Por cada variante, crear un método en cada subclase que redefina el de la superclase.

3. Copiar al método de cada subclase la parte del condicional correspondiente.

4. Compilar y testear.

5. Borrar de la superclase la sección (branch) del condicional que se copió.
6. Compilar y testear.

7. Repetir para todos los branchs del condicional.
8. Hacer que el método de la superclase sea abstracto.

## Simplificación en la invocación de métodos:

- Sirven para mejorar la interfaz de una clase

### Rename Method

- Motivación: Mejorar la claridad del código. Un buen nombre explica su propósito.
- Mecánica:
    1. Renombrar el método.
    2. Actualizar todas sus llamadas.

### Preserve Whole Object

- Motivación: Si un método recibe muchos valores de un mismo objeto, conviene pasar el objeto entero.
- Mecánica:
    1. Identificar el objeto del cual provienen los datos.
    2. Modificar el método para recibir el objeto.
    3. Reemplazar las llamadas para pasar el objeto.

### Introduce Parameter Object

- Motivación: Cuando un método recibe muchos parámetros relacionados.

- Mecánica:
    1. Crear una clase para agrupar esos parámetros.
    2. Modificar el método para recibir un solo parámetro de esa clase.

### Parameterize Method

- Motivación: Si hay varios métodos similares que sólo difieren en valores constantes.
- Mecánica:
    1. Unificar los métodos en uno solo.
    2. Agregar un parámetro que indique la variación.
    3. Eliminar los métodos redundantes.

## Manipulación de la generalización:

- Ayudan a mejorar las jerarquías de clases

### Push Up / Down Field

- Motivación: Subir atributos comunes a una superclase o bajar los específicos a subclases
- Mecánica:
    1. Identificar campos duplicados.
    2. Moverlos a la clase correspondiente.

### Push Up / Down Method

- Motivación: Similar al anterior, pero con métodos.
- Mecánica:
    1. Verificar si el método es igual o similar en varias clases.
    2. Si es igual, subirlo. Si es específico, bajarlo.

### Pull Up Constructor Body

- Motivación: Consolidar lógica duplicada en constructores de subclases.
- Mecánica:
    1. Mover código común del constructor a la superclase.

### Extract Subclass / Superclass

- Motivación: Separar responsabilidades o extraer comportamiento común.
- Mecánica:
    1. Crear una nueva clase hija o padre.
    2. Mover atributos o métodos correspondientes.

### Collapse Hierarchy

- Motivación: Cuando una jerarquía es innecesaria o la subclase no agrega valor.
- Mecánica:
    1. Mover todo a una sola clase.
    2. Eliminar la clase innecesaria.

### Replace Inheritance with Delegation

- Motivación: Cuando la subclase no necesita todo lo de la superclase o hay acoplamiento innecesario.
- Mecánica:
    1. Crear una instancia de la superclase como atributo.
    2. Delegar los métodos necesarios.
    3. Eliminar la herencia.

### Replace Delegation with Inheritance

- Motivación: Cuando una clase está delegando todo y podría heredar directamente.
- Mecánica:
    1. Hacer que la clase extienda la otra.
    2. Eliminar la delegación

**Pull Up Method**

1. Asegurarse que los métodos sean idénticos. Si no, parametrizar
2. Si el selector del método es diferente en cada subclase, renombrar
3. Si el método llama a otro que no está en la superclase, declararlo como abstracto en la superclase
4. Si el método llama a un atributo declarado en las subclases, usar “Pull Up Field” o “Self Encapsulate Field” y declarar los getters abstractos en la superclase
5. Crear un nuevo método en la superclase, copiar el cuerpo de uno de los métodos a él, ajustar, compilar
6. Borrar el método de una de las subclases
7. Compilar y testear
8. Repetir desde 6 hasta que no quede en ninguna subclase

- Big refactorings

# PATRONES

## Definición general:

- Un patrón es un par problema-solución
- Los patrones tratan con problemas recurrentes y buenas soluciones (probadas) a esos problemas
- La solución es suficientemente genérica para poder aplicarse de diferentes maneras

### Más definiciones

Según Alexander:

> "Un patrón describe un problema que ocurre una y otra vez en un contexto, y además, describe el núcleo de la solución a ese problema de tal forma que pueda ser usada millones de veces sin tener que hacer lo mismo dos veces."

Según Gang of Four:

> "Un patrón de diseño nombra, abstrae e identifica los aspectos clave de una estructura de diseño común que lo hace útil para crear un diseño orientado a objetos reutilizable.
> El patrón de diseño identifica las clases e instancias participantes, sus roles y colaboraciones, y la distribución de responsabilidades.
> Cada patrón de diseño se centra en un problema o cuestión particular del diseño orientado a objetos. Describe cuándo se aplica, si puede aplicarse en vista de otras restricciones de diseño y las consecuencias y compensaciones de su uso."

## Características

- Cada patrón del catálogo describe una solución simple y elegante a un problema específico en el diseño de software orientado a objetos (OO).
- Los patrones capturan soluciones que han sido desarrolladas y evolucionadas en el tiempo, a partir del trabajo de muchos

desarrolladores: rediseñar, fallar, reflexionar. Cada patrón:

- Nombra, abstrae e identifica los aspectos claves de un diseño que resuelve un problema particular:
    - Clases que participan
    - Roles
    - Relaciones
    - Distribución de responsabilidades
    - Consecuencias, pros y contras

## Elementos de un patrón

- Nombre
- Propósito
- Motivación (Problema)
- Aplicabilidad

- Estructura
- Participantes
- Colaboraciones
- Consecuencias
- Implementación
- Código
- Usos conocidos
- Patrones relacionados

## Clasificaciones

**Según la actividad:**

- Patrones de Software
- Patrones de Testing
- Patrones de Proceso
- Patrones Pedagógicos

**Según el nivel de abstracción:**

- Patrones de Análisis
- Patrones de Arquitectura
- Patrones de Diseño

- *Idioms*

**Según el dominio de aplicación:**

- Financiero
- Comercio electrónico
- Salud
- Tiempo Real

**Según el ambiente/paradigma:**

- Patrones Web
- Patrones de Modelos de Datos
- XML
- Interacción

**De acuerdo al propósito:**

- Creacionales
- Estructurales
- De Comportamiento.

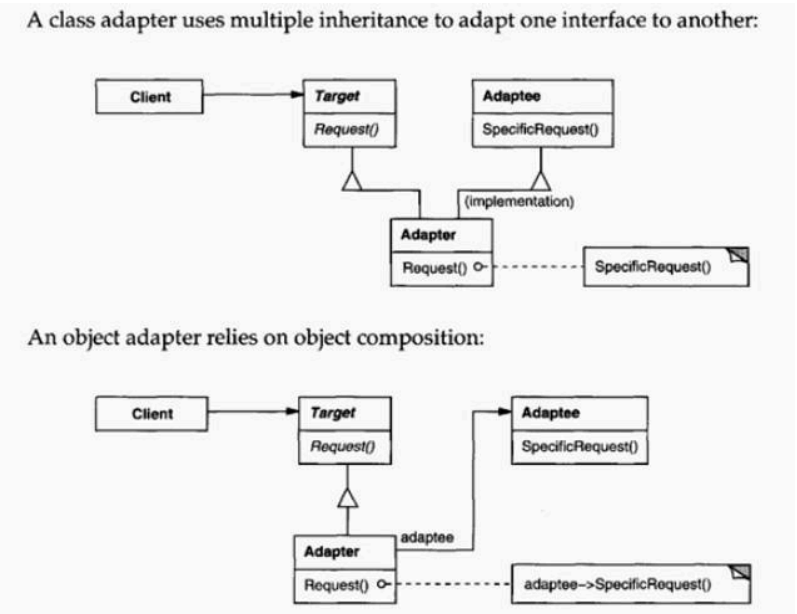
## PATRONES ESTRUCTURALES

### **ADAPTER** / Wrapper (Envoltorio)

- **Intención**:
    - "Convertir" la interfaz de una clase en otra que el cliente espera. Adapter permite que ciertas clases con interfaces incompatibles puedan trabajar en conjunto
- **Motivación**
    - En algunos casos, clases existentes no pueden reutilizarse directamente porque su interfaz es incompatible con la requerida por una aplicación.
    - Modificar estas clases no siempre es posible ni conveniente, especialmente si forman parte de bibliotecas externas.

- La solución consiste en crear una clase **adaptadora** que haga de puente entre la interfaz esperada y la interfaz real de la clase existente. Esta adaptación puede hacerse:
    - Heredando de ambas interfaces.
    - Componiendo un objeto interno y delegando las llamadas.

- **Aplicabilidad**:
    - Use Adapter cuando Ud. quiere usar una clase existente y su interfaz no es compatible con lo que precisa
- **Estructura**



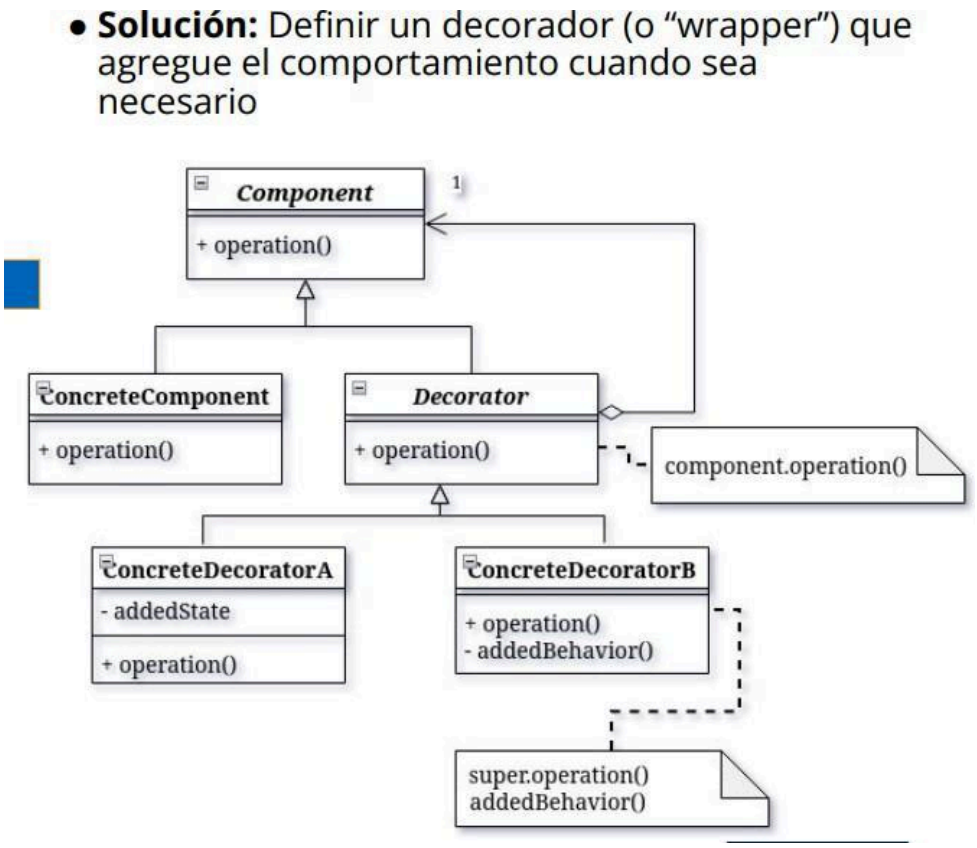

- **Participantes**
    - Target: Define la interfaz específica que usa el cliente
    - Client: Colabora con objetos que satisfacen la interfaz de Target
    - Adaptee: Define una interfaz que precisa ser adaptada
    - Adapter: Adapta la interfaz del Adaptee a la interfaz de Target
- **Colaboraciones**
    - Los objetos Client llaman a las operaciones en la instancia del Adapter
    - A su vez, el Adapter llama a las operaciones definidas en el Adaptee
- **Consecuencias**
    - Una misma clase Adapter puede usarse para muchos Adaptees (el Adaptee y todas sus subclases)
    - El Adapter puede agregar funcionalidad a los adaptados
    - Se generan más objetos intermediarios
- **Implementación**:

    - pluggable adapters, parameterized adapters
- **Patrones relacionados:**
    - El patrón Decorator decora otro objeto sin cambiar su interfaz. Un decorador es por tanto más transparente a la aplicación que un adaptador. Como resultado, el patrón Decorator permite la composición recursiva, lo que no es posible con adapters.
    - El patrón Proxy define un representante o sustituto de otro objeto sin cambiar su interfaz.

**DECORATOR** /Wrapper (Envoltorio)

- **Intención:** Agregar comportamiento a un objeto dinámicamente y en forma transparente.
- **Problema**:
    - Cuando queremos agregar comportamiento adicional a ciertos objetos de una clase, una opción es usar herencia. Sin embargo, presenta limitaciones cuando necesitamos que ese **comportamiento se pueda agregar o quitar dinámicamente en tiempo de ejecución**.
    - Cuando la herencia no es adecuada, ya que implica una decisión estática. Esta rigidez hace que la herencia no sea flexible para escenarios donde los objetos necesitan "mutar de clase" o modificar su comportamiento de forma dinámica.
- **Motivación**:
    - A veces queremos agregar responsabilidades a objetos específicos sin modificar toda la clase. La herencia es inflexible para esto.
    - **Decorator** permite envolver el objeto en otro que añade funcionalidades (como bordes o desplazamiento) sin cambiar su interfaz.
    - Esto permite agregar comportamientos de forma dinámica y combinarlos libremente.
- **Aplicabilidad**
    - Agregar responsabilidades a objetos individualmente y en forma transparente (sin afectar otros objetos)
    - Quitar responsabilidades dinámicamente
    - Cuando subclasificar es impráctico.

- **Estructura**:




- **Participantes**
  - Component: Define la interfaz para objetos a los que se puede añadir responsabilidades dinámicamente.
  - Concrete component: Define un objeto al que se pueden añadir responsabilidades adicionales.
  - Decorator:Mantiene una referencia a un objeto Componente y define una interfaz que se ajusta a la interfaz del Componente.
  - Concrete decorator: Añade responsabilidades al componente.

- **Colaboraciones**
  - El Decorador redirige peticiones a su objeto Componente. Opcionalmente puede realizar operaciones adicionales antes y después de reenviar la petición.
- **Consecuencias**
  - Puntos a favor:
    - Permite mayor flexibilidad que la herencia.
    - Permite agregar funcionalidad incrementalmente.

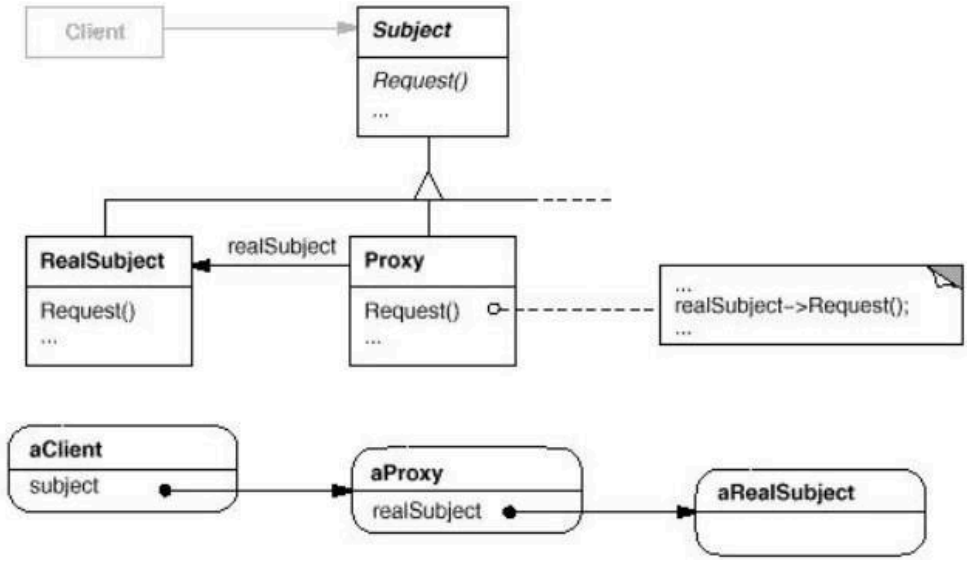
    - Puntos en contra
        - Mayor cantidad de objetos
        - complejo para depurar

- **Implementación**
    - Misma interface entre componente y decorador.
    - No hay necesidad de la clase Decorator abstracta, si se tiene un solo decorador.
    - Cambiar el "skin" vs cambiar sus "guts"
        - Decorator puede verse como una "piel" que modifica el comportamiento externo, no la estructura interna (los "guts").
        - Es decir: no cambiamos su estructura interna

- **Patrones relacionados**
    - Adapter:
        - Ambos patrones "decoran" el objeto para cambiarlo.
        - El Decorator preserva la interface del objeto para el cliente.
        - Adapter convierte la interface del objeto para el cliente.
        - Decorators pueden y suelen anidarse.
        - Adapters no se anidan.
    - Composite:
        - El decorator mantiene una estructura con sólo un siguiente
        - El composite mantiene una estructura de tipo árbol, un composite usualmente se compone de varias partes
        - El propósito del decorator es agregar funcionalidad dinámicamente
        - El propósito del composite es componer objetos y tratarlos de manera uniforme
    - Strategy:
        - ¿En qué se parecen? → Propósito: permitir que un objeto cambie su funcionalidad dinámicamente (agregando o cambiando el algoritmo que utiliza)
            - El Decorator cambia el algoritmo por fuera del objeto
            - El Strategy cambia el algoritmo por dentro del objeto,

**PROXY** /Surrogate (Sustituto)

- **Intención**:proporcionar un intermediario de un objeto para controlar su acceso. TAMBIÉN CONOCIDO COMO
  Surrogate (Sustituto)
- **Motivación**:
  - Una de las razones para controlar el acceso a un objeto es posponer el costo completo de su creación e inicialización hasta que realmente necesitemos usarlo.
- **Aplicabilidad**: cuando se necesita una referencia más flexible hacia un objeto Aplicaciones del proxy:
  - ➔ Virtual proxy: demorar la construcción de un objeto hasta que sea realmente necesario, cuando sea poco eficiente acceder al objeto real.
  - ➔ Protection proxy: Restringir el acceso a un objeto por seguridad
  - ➔ Remote proxy: representar un objeto remoto en el espacio de memoria local. Es la forma de implementar objetos distribuidos. Estos proxies se ocupan de la comunicación con el objeto remoto, y de serializar/deserializar los mensajes y resultados.
    - ◆ Para acceder a objetos que se encuentran en otro espacio de memoria, en una arquitectura distribuida
    - ◆ El proxy empaqueta el request, lo envía a través de la red al objeto real, espera la respuesta, desempaqueta la respuesta y retorna el resultado
    - ◆ En este contexto el proxy suele utilizarse con otro objeto que se encarga de encontrar la ubicación del objeto real. Este objeto se denomina Broker, del patrón de su mismo nombre
- **Estructura**
  - Colocar un objeto intermedio que respete el protocolo del objeto que está reemplazando.
  - Algunos mensajes se delegarán en el objeto original. En otros casos puede que el proxy colabore con el objeto original o que reemplace su comportamiento




- **Participantes**:
  - Proxy:
    - Mantiene una referencia que permite al proxy acceder al objeto real
    - Proporciona una interfaz idéntica a la de Sujeto, de manera que un proxy pueda ser sustituido por el sujeto real
    - Controla el acceso al sujeto real y puede ser responsable de su creación y borrado
    - Otras responsabilidades dependen del tipo de proxy:

      - Los <u>proxies remotos</u> son responsables de codificar una petición y sus argumentos para enviar la petición codificada al sujeto real que se encuentra en un espacio de direcciones diferente
      - Los <u>proxies virtuales</u> pueden guardar información adicional sobre el sujeto real, por lo que pueden retardar el acceso al mismo
      - Los <u>proxies de protección</u> comprueban que el llamador tenga los permisos de acceso necesarios para realizar una petición.

  - Subject:Define la interfaz común para el Real Subject y el Proxy, de modo que pueda usarse un Proxy en cualquier sitio en el que se espere un Real Subject
  - Real Subject:Define el objeto real representado.

- **Colaboraciones:**
  - El Proxy redirige peticiones al SujetoReal cuando sea

necesario, dependiendo del tipo de proxy.

- **Consecuencias**:
  - Puntos a favor
    - Indirección en el acceso al objeto: El patrón Proxy introduce un nivel de indirección que permite controlar el acceso al objeto real, lo que ofrece flexibilidad en su uso
  - Puntos en contra
    - Complejidad adicional
- **Patrones relacionados**
  - Adapter:
    - El proxy proporciona la misma interfaz que su sujeto
    - El adapter proporciona una interfaz diferente al objeto que adapta.
  - Decorator:
    - El proxy controla el acceso a un objeto.
    - El decorator agrega una o más responsabilidades a un objeto,

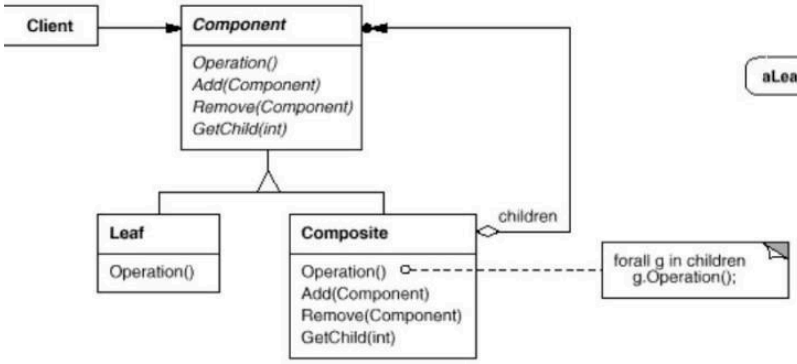
**Adapter, Decorator, Proxy**

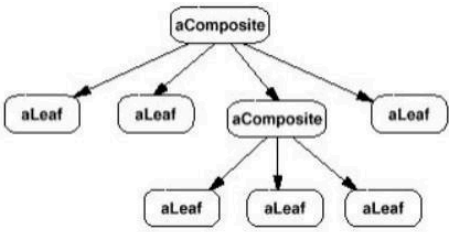
- Todos patrones estructurales
- Todos con diagramas de objetos similares
- Distinto propósito
- A todos se los llama también "wrappers"

## COMPOSITE

- **Intención**: Componer objetos en estructuras de árbol para representar jerarquías parte-todo. El Composite permite que los clientes traten a los objetos atómicos y a sus composiciones uniformemente.
- **Motivación** Cuando una aplicación debe manejar objetos individuales y agrupaciones de objetos, tratarlos por separado complica el diseño. El patrón Composite propone una estructura jerárquica donde ambos tipos comparten una interfaz común, permitiendo que el código cliente los trate de forma uniforme y más simple.
- **Aplicabilidad**: Use el patrón Composite cuando
  - Quiere representar jerarquías parte-todo de objetos.
  - Quiere que los objetos "clientes" puedan ignorar las diferencias entre composiciones y objetos individuales. Los clientes trataran a los objetos atómicos y compuestos uniformemente

- Estructura:




- **Participantes**:
  - Component:
    - Declara la interfaz para los objetos de la composición.
    - Implementa comportamientos default para la interfaz común a todas las clases.
    - Declara la interfaz para definir y acceder "partes de la composición".
  - Leaf:
    - Las hojas no tienen sub-árboles.
    - Define el comportamiento de objetos primitivos en la composición.

    Composite:
    - Define el comportamiento para componentes complejos.
    - Implementa operaciones para manejar el sub-árboles
- **Colaboraciones**:
  - Los Clientes usan la interfaz de la clase Component para interactuar con los objetos de la estructura compuesta.
    - Si el recipiente es una Leaf, la petición se trata correctamente.
    - Si es un Composite, normalmente redirige las peticiones a sus componentes hijos, posiblemente realizando operaciones adicionales antes o después.
- **Consecuencias:**
  - Positivas:
    - Define "jerarquías" de objetos primitivos y compuestos.
    - Los objetos primitivos pueden componerse en objetos complejos, los que a su vez pueden componerse recursivamente.
    - Simplifica los objetos cliente. Los clientes usualmente no

saben (y no deberían preocuparse) acerca de si están manejando un compuesto o un simple.
        - Hace más fácil el agregado de nuevos tipos de componentes porque los clientes no tienen que cambiar cuando aparecen nuevas clases componentes.
    - Negativas:
        - Debe ser "customizado" con reglas de composición (si fuera necesario)

- **Implementación**:
    - Referencias explícitas a la raíz de una hoja
    - Maximizar el protocolo de la clase/interfaz Component
    - Orden de las hojas
    - Borrado de componentes
    - Búsqueda de componentes(fetch) por critérios
    - Diferentes estructuras de datos para guardar componentes

- **Patrones relacionados**:
  El patrón Decorator suele usarse junto con el Composite. Cuando se usan juntos decoradores y compuestos, normalmente ambos tendrán una clase padre común. Por tanto, los decoradores tendrán que admitir la interfaz Component con operaciones como Añadir, Eliminar y ObtenerHijo.

- **Errores comunes:** La jerarquía no es polimórfico Composite no tiene relación con otras clases de Jerarquía Composite no implementa add/remove Composite no delega en la colección.

---

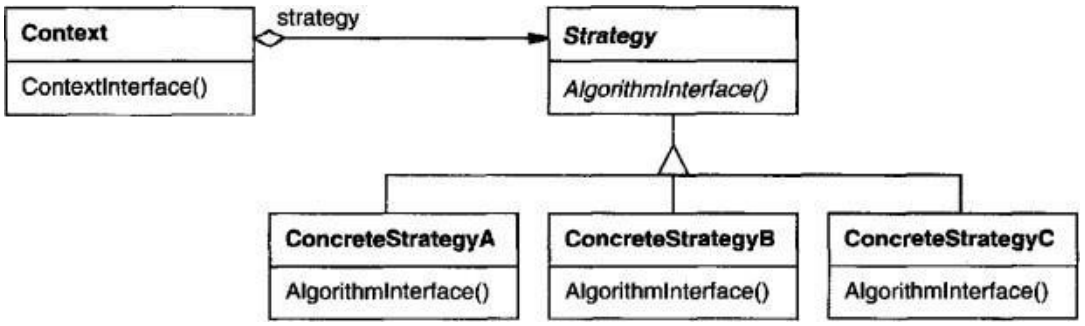
## PATRONES DE COMPORTAMIENTO

### **STRATEGY** / Policy (Política)

- **Intención**:
  - Definir una familia de algoritmos, encapsular cada uno y hacerlos intercambiables.
  - Permite que el algoritmo varíe independientemente de los clientes que lo usan.
  - Permitir cambiar (en forma dinámica), el algoritmo que se utiliza.
  - Brindar flexibilidad para agregar nuevos algoritmos que lleven a cabo una función determinada

- **Motivación**
  - Existen muchos algoritmos para dividir en líneas un flujo de texto.
  - Codificar dichos algoritmos en las clases que los usan no resulta una buena práctica por varias razones

- **Aplicabilidad**:
  Uso el patrón Strategy cuando
  - Existen muchos algoritmos para llevar a cabo una tarea.
  - No es deseable codificarlos todos en una clase y seleccionar cuál utilizar por medio de sentencias condicionales.
  - Cada algoritmo utiliza información propia. Colocar esto en los clientes lleva a tener clases complejas y difíciles de mantener.
  - Es necesario cambiar el algoritmo en forma dinámica, en tiempo de ejecución.
- **Estructura**
  - Son los clientes del contexto los que generalmente crean las estrategias.




- **Participantes**
  - Strategy: Declara una interfaz común a todos los algoritmos permitidos. Context usa esta interfaz para llamar al algoritmo definido por una ConcreteStrategy.

    - Concrete strategy: Implementa el algoritmo usando la interfaz Strategy.
    - Context: se configura con un objeto ConcreteStratey.
        - mantiene una referencia a un objeto Strategy.
        - puede definir una interfaz que permita a la Strategy acceder a sus datos.
- **Colaboraciones**
    - Strategy y Context interactúan para implementar el algoritmo elegido.
    - Un Context puede pasar en la estrategia todos los datos requeridos por el algoritmo cada vez que se llama a éste.
    - Otra alternativa es que el Context se pase a sí mismo como argumento de las operaciones de Strategy. Eso permite a Strategy hacer llamadas al contexto cuando sea necesario.
    - Un Context redirige peticiones de los clientes a su estrategia.
    - Los Client normalmente crean un objeto ConcreteStrategy, el cual pasan al Context; por tanto, los Client interactúan exclusivamente con el Context.
    - Suele haber una familia de clases ConcreteStrategy a elegir por el cliente.

- **Consecuencias**:
    - Puntos a favor
        - Mejor solución que subclasificar el contexto, cuando se necesita cambiar dinámicamente.
        - Desacopla al Context de los detalles de implementación de las Strategy.
        - Se eliminan los condicionales.
    - Puntos en contra
        - La clase cliente debe conocer las diferentes estrategias para poder elegir.
        - Overhead en la comunicación entre Context y ConcreteStrategy.

- **Implementación**:
    - El Context debe tener métodos en su protocolo que permitan cambiar la ConcreteStrategy
    - Parámetros entre el Contexty la Strategy
        - Hay que analizar qué datos se necesitan particularmente en cada caso

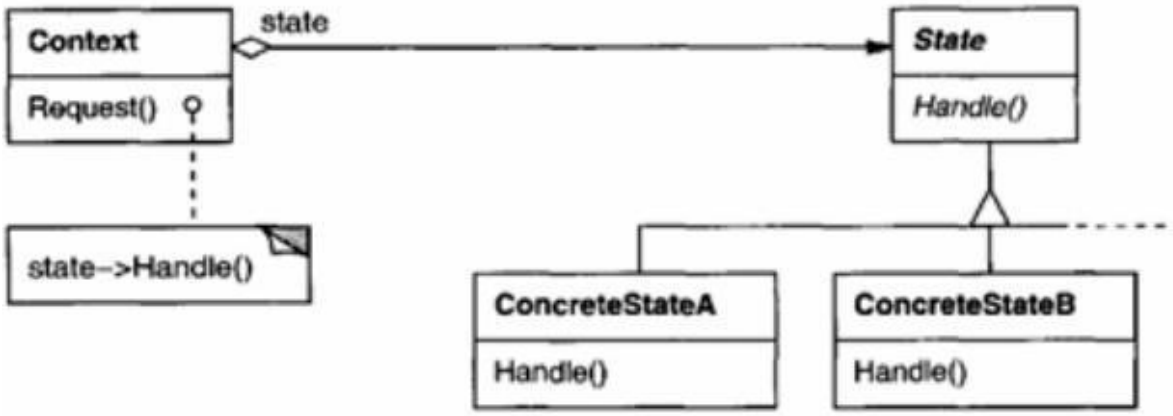
## **STATE** / Objects for States (Estados como Objetos)

- **Intención**:
  - Modificar el comportamiento de un objeto cuando su estado interno se modifica.
  - Externamente parecería que la clase del objeto ha cambiado

- **Motivación**:
  - Cuando un objeto debe cambiar su comportamiento según su estado interno, el patrón **State** permite encapsular estos comportamientos en clases separadas.
  - Cada estado se representa como una subclase que define el comportamiento correspondiente, evitando condicionales y mejorando la organización del código.

- **Aplicabilidad**:Use el patrón State cuando
  - El comportamiento de un objeto depende del estado en el que se encuentre.
  - Los métodos tienen sentencias condicionales complejas que dependen del estado.
  - Este estado se representa usualmente por constantes enumerativas y en muchas operaciones aparece el mismo condicional. El patrón State reemplaza el condicional por clases **(es un uso inteligente del polimorfismo)**

- **Detalles**:
  - Desacoplar el estado interno del objeto en una jerarquía de clases.
  - Cada clase de la jerarquía representa un estado concreto en el que puede estar el objeto.
  - Todos los mensajes del objeto que dependan de su estado interno son delegados a las clases concretas de la jerarquía (polimorfismo).
- **Estructura**




- **Participantes**
  - Context:
    - Define la interfaz que conocen los clientes.
    - Mantiene una instancia de alguna clase de ConcreteState que define el estado corriente
  - Strate:
    - Define la interfaz para encapsular el comportamiento de los estados de Context
  - ConcreteState:
    - Cada subclase implementa el comportamiento respecto al estado específico.

- **Colaboraciones**:
  - Contexto delega las peticiones que dependen del estado en el objeto ConcreteState actual.
  - Un Context puede pasarse a sí mismo como parámetro para que el objeto State maneje la petición. Esto permite al objeto State acceder al Context si fuera necesario.
  - Context es la interfaz principal para los clientes. Los clientes pueden configurar un contexto con objetos State. Una vez que está configurado el contexto, sus clientes ya no tienen que tratar con los objetos State directamente.
  - **Cualquiera de las subclases de Context o de ConcreteState pueden decidir qué State sigue a otro y bajo qué circunstancia**

- **Consecuencias**:
    - Puntos a favor
        - Localiza el comportamiento relacionado con cada estado.
        - Las transiciones entre estados son explícitas.
        - En el caso que los estados no tengan variables de instancia pueden ser compartidos.
    - Puntos en contra
        - En general hay bastante acoplamiento entre las subclases de State porque la transición de estados se hace entre ellas, por lo que deben conocerse entre sí

- **Implementacion**:
    - Temas Interesantes
        - Los estados son internos al contexto

- **Patrones relacionados:**
    - Strategy:
        - En el Strategy necesito uno de diferentes algoritmos opcionales para realizar una misma tarea
        - En el state el comportamiento de un objeto depende del estado en el que se encuentre
        - El patrón Strategy es útil para permitir que una clase delegue la ejecución de un algoritmo a una instancia de una familia de estrategias
        - El patrón State es útil para una clase que debe realizar transiciones entre estados fácilmente.
        - En State, los diferentes estados:
            - son internos al contexto
            - no los eligen las clases clientes.
            - la transición se realiza entre los estados mismos
        - En Strategy, las diferentes estrategias:
            - son conocidas desde afuera del contexto, por las clases clientes del contexto
            - el Contexto del Strategy debe contener un mensaje público para cambiar el ConcreteStrategy
        - El State es privado del objeto, ningún otro objeto sabe de él. vs. ≠El Strategy suele setearse por el cliente, que debe conocer las posibles estrategias concretas.

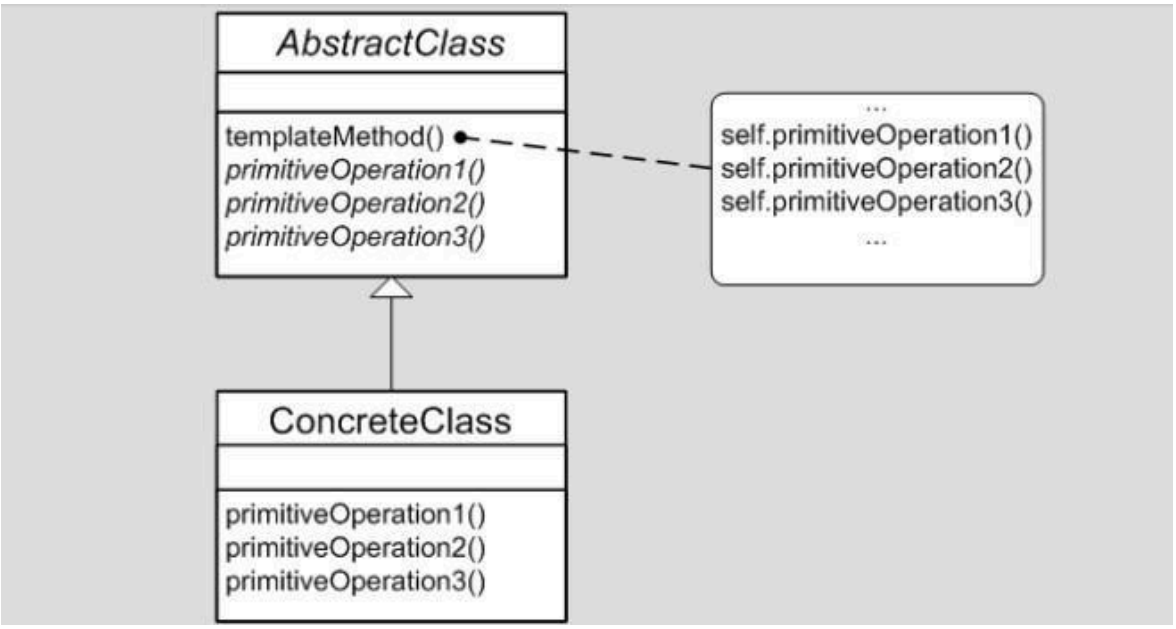
        - Cada State puede definir muchos mensajes. vs. ≠Un Strategy suele tener un único mensaje público.
        - Los states concretos se conocen entre sí. Saben a cual estado se debe pasar en respuesta a algún mensaje. ≠Los strategies concretos no.

**TEMPLATE METHOD:**

- **Intención**:
    - Definir el esqueleto de un algoritmo en un método, difiriendo algunos pasos a las subclases.
    - Template Method permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar la estructura del algoritmo
- **Motivación**:
    - Cuando se necesita definir el esqueleto de un algoritmo pero permitiendo que ciertos pasos sean redefinidos por subclases, el patrón **Template Method** permite fijar la estructura general en una clase base y delegar detalles a las subclases.
    - **Esto favorece la reutilización y la extensión del comportamiento sin alterar la lógica general.**

- **Aplicabilidad**: usar Template Method:
    - Para implementar las partes invariantes de un algoritmo una vez y dejar que las subclases implementen los aspectos que varian
    - Para evitar duplicación de código entre subclases
    - Para controlar las extensiones que pueden hacer las subclases

- **Estructura**:




- **Participantes**
  - AbstractClass: Implementa un método que contiene el esqueleto de un algoritmo (el template method). Ese método llama a operaciones primitivas así como operaciones definidas en AbstractClass Declara operaciones primitivas abstractas que las subclases concretas deben definir para implementar los pasos de un algoritmo
  - ConcreteClass: Implementa operaciones primitivas que llevan a cabo los pasos específicos del algoritmo

- **Colaboraciones**
  - ConcreteClass confía en que la superclase implemente las partes invariantes del algoritmo

- **Consecuencias**:
  - Técnica fundamental de **reuso de código**
  - Lleva a tener **inversión de control** (la superclase llama a las operaciones definidas en las subclases)
  - El template method llama a dos tipos de operaciones:
    - operaciones primitivas (abstractas en AbstractClass y que las subclases tienen que definir)
    - operaciones concretas definidas en AbstractClass y que las subclases pueden redefinir si hace falta (hook methods)

- **Implementación**:
    - minimizar la cantidad de operacione primitivas que las subclases deben redefinir

---

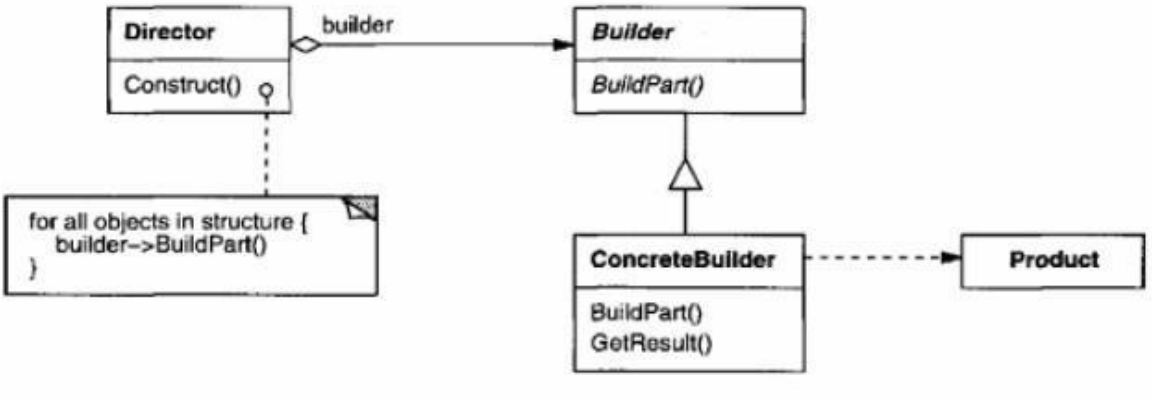
## PATRONES CREACIONALES

### BUILDER

- **Intención**: separa la construcción de un objeto complejo de su representación (implementación) de tal manera que el mismo proceso puede construir diferente representaciones (implementaciones)
- **Motivación**:
    - El patrón **Builder** separa la  de un documento de la construcción de su representación.
    - Permite usar el mismo algoritmo de procesamiento con distintos constructores, facilitando la creación de múltiples salidas sin modificar el código principal.

- **Aplicabilidad** Úsese el patrón Builder cuando
    - El algoritmo para crear un objeto complejo debiera ser independiente de las partes de que se compone dicho objeto y de cómo se ensamblan.
    - El proceso de construcción debe permitir diferentes representaciones del objeto que está siendo construido.

- **Estructura**:




- **Participantes**
  - Builder: especifica una interface abstracta para crear partes de un Producto
  - Concrete Builder: construye y ensambla partes del producto.
    - Guarda referencia al producto en construcción.
  - Director: Conoce los pasos para construir el objeto
    - Utiliza el Builder para construir las partes que va ensamblando ○ En lugar de pasos fijos puede seguir una "especificación" (ver knwon uses@ GOF)
  - Product: es el objeto complejo a ser construido

- **Colaboraciones**
  - El cliente crea el objeto Director y lo configura con el objeto
  - Constructor deseado.
  - El Director notifica al constructor cada vez que hay que construir una parte de un producto.
  - El Constructor maneja las peticiones del director y las añade al producto.
  - El cliente obtiene el producto del constructor.

- **Consecuencias**
    - Puntos a favor:
        - Abstrae la construcción compleja de un objeto complejo
        - Permite variar lo que se construye Director <-> Builder
        - Da control sobre los pasos de construcción
    - Puntos en contra:
        - Requiere diseñar y implementar varios roles
        - Cada tipo de producto requiere un ConcreteBuilder
        - Builder suelen cambiar o son parsers de specs (> complejidad)

- **Consideraciones sobre Builder**
    1. El Director solo sabe hacer una cosa
    2. Los Builders pueden saber hacer cosas que no requiera un Director pero si otro
    3. Los Builders funcionan (generalmente) "injectando dependencias" para armar configuraciones en run-time.
    4. Otros Directores pueden usar los mismos Builders.

- **Errores comunes**:
    - Director construye partes
    - Director necesita diferentes builders al mismo tiempo
    - Builder manda mensajes a Director
    - Solo hay un Builder

- **Relación con otros patrones**:
    - Muchas veces lo que construye el constructor es un Composite .

---

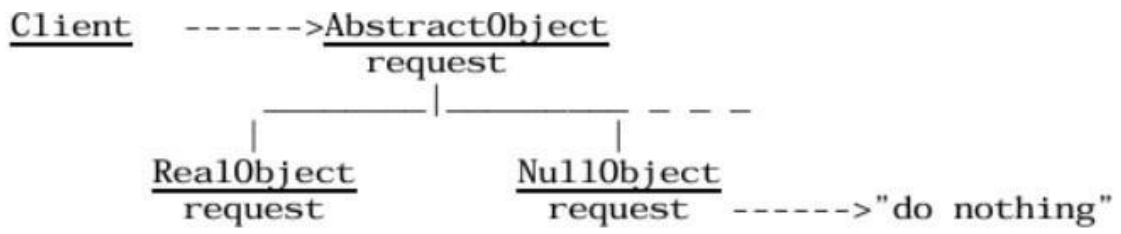
//No sé de que categoria son el null object ni el type object

## NULL OBJECT

- **Intención**:Proporcionar un sustituto para otro objeto que comparte la misma interfaz, pero no hace nada. El objeto nulo encapsula las decisiones de implementación sobre cómo "no hacer nada" y oculta esos detalles a sus colaboradores.

- **Aplicabilidad**: Un objeto tiene un colaborador, que algunas veces no hace nada queremos que el objeto cliente pueda ignorar la diferencia del colaborador que hace algo con el que no hace nada queremos **reusar el comportamiento de hacer nada**

- **Estructura**:




- Participantes:
  - Client
    - Requiere un colaborador
  - Abstract Object
    - Declara la interfaz para el colaborador del Cliente.
    - Implementa el comportamiento predeterminado para la interfaz común a todas las clases, según corresponda.
  - Real Object
    - Define una subclase concreta de AbstractObject cuyas instancias proporcionan un comportamiento útil que el Cliente espera.
  - Null Object
    - Proporciona una interfaz idéntica a la de AbstractObject para que un objeto nulo pueda sustituirse por un objeto real. Implementa su interfaz para no hacer nada.
      - Lo que significa exactamente no hacer nada es subjetivo y depende del tipo de comportamiento que espera el Cliente.
    - Algunas solicitudes pueden cumplirse haciendo algo que da un resultado nulo.
    - Cuando hay más de una forma de no hacer nada, es posible que se requiera más de una clase NullObject.

- **Consecuencias**

- Define jerarquías de clase que consisten de objetos reales y null objects.
- En cualquier lugar que el cliente espera un objeto real también puede tomar un null object
- Hace el código del cliente más simple. Los clientes pueden tratar a sus colaboradores de manera uniforme
- Encapsula el comportamiento de hacer nada en un objeto, que puede ser reusado por múltiples clientes
- Requiere crear una clase NullObject cada vez que en una jerarquía necesitemos el comportamiento nulo
- Podría ser difícil de implementar si la usan varios clientes y no hay un acuerdo de cuál debería ser el comportamiento nulo

# REFACTORING TO PATTERNS

## ¿Cuándo usamos patrones?

### Consideraciones económicas

- **Kent Beck** – *"Economic imperatives"*:
  - **Survival (supervivencia)**
  - **Net Present Value**: ganar antes, gastar después
  - **Optionality**: agregar valor es agregar opciones

- **Joshua Kerievsky** – *"Over-engineering is as dangerous as under-engineering"*:
  - **Over-engineering**: construir software más sofisticado de lo necesario.
  - **Under-engineering**: construir software con diseño pobre.

### Over-Engineering

**¿Por qué se hace?**

- Para acomodar futuros cambios (pero no se puede predecir el futuro).
- Para evitar un mal diseño (aunque los patrones pueden hacernos perder de vista soluciones simples).

**Consecuencias**

- Código sofisticado y complejo que complica el mantenimiento.
- Nadie lo entiende ni lo quiere tocar.
- Se generan duplicaciones por falta de comprensión.

## La panacea de los patrones

- Son tentadores para evitar malos diseños.
- También pueden llevar al exceso.
- **Importante**: conocer las consecuencias positivas y negativas.

## ¿Cómo ayuda el *refactoring*?

- Cuando el código ya funciona y pasa los tests.
- Provee mecanismos para resolver problemas de diseño.
- A través de cambios pequeños:
  - Más fáciles y seguros que un gran cambio.

- Permiten testear tras cada cambio.
- Cada cambio revela otros posibles.

## Refactoring to Patterns

- Nos permite introducir patrones solo cuando son necesarios.
- No hace falta adivinar ni prever

### Form Template Method

**Propósito del patrón**

- Definir el **esqueleto de un algoritmo** y diferir pasos a subclases.
- Permite redefinir pasos **sin cambiar la estructura general**.

**Problema que soluciona**

- **Reduce/elimina código repetido** en subclases.

**Refactoring "Form Template Method"**

- Dos o más métodos realizan pasos similares en el mismo orden, pero con diferencias.

**Mecánica**

1. Identificar métodos similares y extraer partes: métodos idénticos o únicos.
2. Pull Up Method para los métodos idénticos.
3. Rename Method en los únicos hasta que queden idénticos.
4. Compilar y testear después de cada rename.
5. Rename Method a los métodos esqueleto.
6. Pull Up Method a métodos similares.
7. Definir métodos abstractos en la superclase.
8. Compilar y testear.

Pros

- Elimina código duplicado.
- Simplifica y comunica el algoritmo.
- Las Subclases pueden adaptar fácilmente.

Contras

- Complica el diseño si las subclases deben implementar muchos métodos.

## Extract Adapter

**Motivación**

- Una clase contiene lógica que traduce o adapta llamadas hacia otra clase que tiene una interfaz diferente.
- Esto genera acoplamiento innecesario, hace difícil el testing y mezcla responsabilidades (violación de SRP).

**Solución**:

- Con Extract Adapter, esa lógica de traducción se mueve a una clase Adapter independiente.

**Mecánica**:

1. Identificar el código de adaptación incrustado. Ejemplo:
    - Conversión de nombres de métodos
    - Transformación de formatos
    - Traducción de tipos o de estructuras
2. Crear nueva clase adapter
    - La nueva clase tendrá como dependencia (composición) al objeto que originalmente recibía las llamadas adaptadas.
3. Mover la lógica de adaptación
    - Extraer el código de traducción y moverlo al Adapter mediante Move Method
    - El Adapter expone los métodos en la interfaz esperada por el cliente
4. Modificar el cliente para que use el Adapter
5. Eliminar dependencias innecesarias
    - Eliminar todas las referencias a la clase adaptada.
6. Compilar y testear después de cada movimiento

**Ventajas**:

- Reduce el acoplamiento entre clases
- Mejora la testeabilidad porque el adapter puede ser simulado (Mock/Stub)
- Refuerza SRP

- Facilita reutilización del código de adaptación

## Replace Implicit Tree with Composite

**Motivación**

- Construir árboles con primitivas como String acopla lógica y representación.
- Dificulta el mantenimiento y la extensión.

**Solución**

- Reemplazar construcción manual por un Composite (ej. XML) con objetos como TagNode.

**Ventajas**

- Mejor comunicación de la estructura.
- Menos duplicación.
- Menos errores.
- Facilita mantenimiento/extensión.

**Mecánica básica**

1. Identificar construcción manual.
2. Diseñar clase de nodo.
3. Permitir hijos y atributos.
4. Reemplazar por el Composite.
5. Compilar, testear, refactorizar.

## Replace Conditional Logic with Strategy

**Motivación**

- Cuando hay lógica condicional que elige variantes

**Qué hacer:**

- Crear un **Strategy** por variante.
- Hacer que el método delegue en el strategy.

**Mecánica**

1. Crear clase Strategy.
2. Move Method: mover lógica del contexto al strategy.

3. Definir v.i. y setter (constructor).
4. Dejar un método delegado en el contexto.
5. Elegir parámetros a pasar.
6. Compilar y testear.
7. Extract Parameter para permitir instanciar el strategy.
8. Replace Conditional with Polymorphism en el strategy.
9. Testear combinaciones de strategy + contexto.

**Alternativa**

Si hay pocas combinaciones:

- Usar Encapsulate Classes with Factory (Kerievsky).
- Crear un método que devuelva el contexto con el strategy ya definido.

## Replace State-Altering Conditionals with State

**Problema**

- Lógica condicional de cambio de estado compleja.

**Qué hacer:**

- Usar objetos State para manejar estados y transiciones.

**Motivación**

- Obtener una vista global de las transiciones.
- Condicionales difíciles de mantener/extender.
- Otros refactorings no alcanzan.

**Mecánica**

1. Replace Type-Code with Class sobre v.i. de estado.
2. Extract Subclass por cada estado concreto.
3. Move Method de la lógica condicional al State.
4. Push Down Method de lógica de transición.
5. Métodos abstractos o por defecto en la superclase State.

## Move Embellishment to Decorator

**Intención**

La intención fundamental de este proceso es mover esos comportamientos a objetos Decorator independientes que permiten añadir responsabilidades a un objeto de forma dinámica y transparente.

**Mecánica**

1. Identificar superclase (o interface) del decorado (Component).
2. Aplicar Replace Conditional Logic with Polymorphism (crear decorator como subclase).
3. Si no alcanza:
    - Replace Inheritance with Delegation (decorator delega).
    - Extract Parameter para asignar decorado.

**Ventajas**

- Flexibilidad frente a la herencia: Los decoradores permiten añadir o quitar responsabilidades en tiempo de ejecución 'envolviendo' los objetos
- Evita clases sobrecargadas
- Refuerza SRP (Solid): Se divide la clases en otras mas pequeñas cohesivas.

## Introduce Null Object

**Fuerzas del problema**

- Muchos condicionales null.
- Queremos tratar null como un objeto más.
- Evitamos errores y excepciones por olvidos.

**Mecánica**

1. Crear null object con Extract Subclass o implementar interface.
2. Buscar null checks en código cliente. Redefinir en el null object.
3. Repetir redefinición para todos los null checks.
4. Reemplazar retornos null por instancias del null object.
5. Eliminar null checks donde ya no son necesarios

# TESTING

## Test de Unidad

- Testeo de la mínima unidad de ejecución.
- En OOP, la mínima unidad es un método
- Objetivo: aislar cada parte de un programa y mostrar que funciona correctamente.
- Cada test confirma que un método produce el output esperado ante un input conocido.
- Es como un contrato escrito de lo que esa unidad tiene que satisfacer.

## Estrategias:

- **Particiones Equivalentes**
  - Considerar el "dominio" de la funcionalidad a testear
    - Si un elemento pasa el test, otros del mismo conjunto pasarán
  - Considerando el complemento del dominio
    - No hace lo que no debe hacer
- **Valores de Borde**
  - Considerando las Particiones Equivalentes
    - Se testean escenarios con valores en los límites de las particiones

## TEST DOUBLE
## Propósito

- Un **Test Double** es un objeto que simula (total o parcialmente) el comportamiento de un DOC para facilitar la prueba del SUT.
- No tiene que replicar toda la funcionalidad del DOC real, solo lo necesario para cada test.
- Reemplazamos un componente del que depende el SUT con un **equivalente específico para pruebas**.

## Definiciones:

SUT y DOC

- **SUT:** *System Under Test* – Sistema Bajo Prueba
  - Es el componente del sistema que se está probando en un determinado caso de prueba.
  - Es la pieza de código que se quiere verificar para asegurarse de que funciona correctamente.

- **DOC**: *Depended-On Component* – Componente del que se depende
  - Cualquier componente que el SUT necesita para funcionar correctamente.
  - Durante una prueba, el SUT interactúa con estos DOCs para llevar a cabo sus operaciones.

## Tipos

### 1. Test Stub

- Cascarón vacío. Sirve para que el SUT envíe los mensajes esperados.
- Objeto que proporciona **respuestas predefinidas** a llamadas del código bajo prueba.
- Simula dependencias reales y puede devolver valores específicos o excepciones controladas.
- Se usa para aislar el código bajo prueba y verificar cómo responde a resultados esperados.

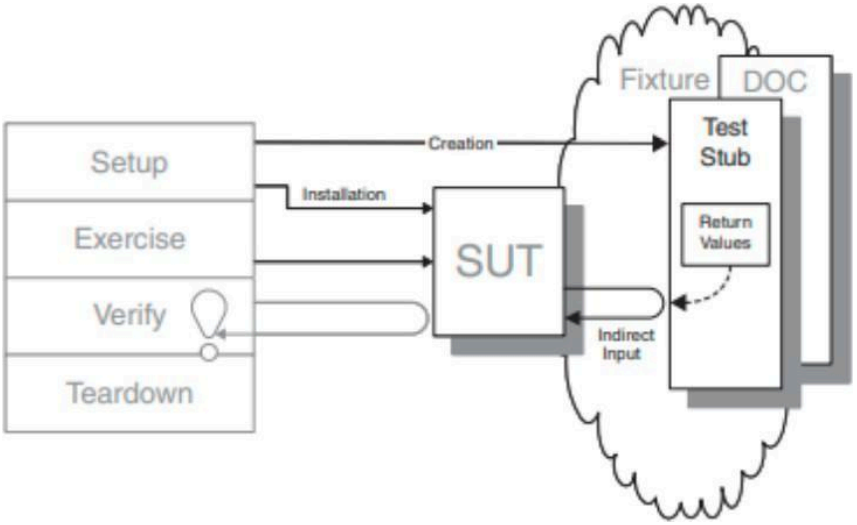
**Propósito: controlar los inputs indirectos que recibe el SUT.**

- Devuelve respuestas predefinidas al SUT.
- No verifica nada, solo "actúa como si".
- Útil para simular valores válidos (**Responder**) o errores (**Saboteur**).
- Reemplaza un objeto real con uno específico para pruebas que proporciona las entradas indirectas deseadas al sistema bajo prueba.

**Cuándo debería usarse:**

- Cuando hay código no probado por no poder controlar las entradas indirectas del SUT.
- Para inyectar valores que permitan superar un punto donde el SUT

llama a software no disponible en nuestro entorno de pruebas.

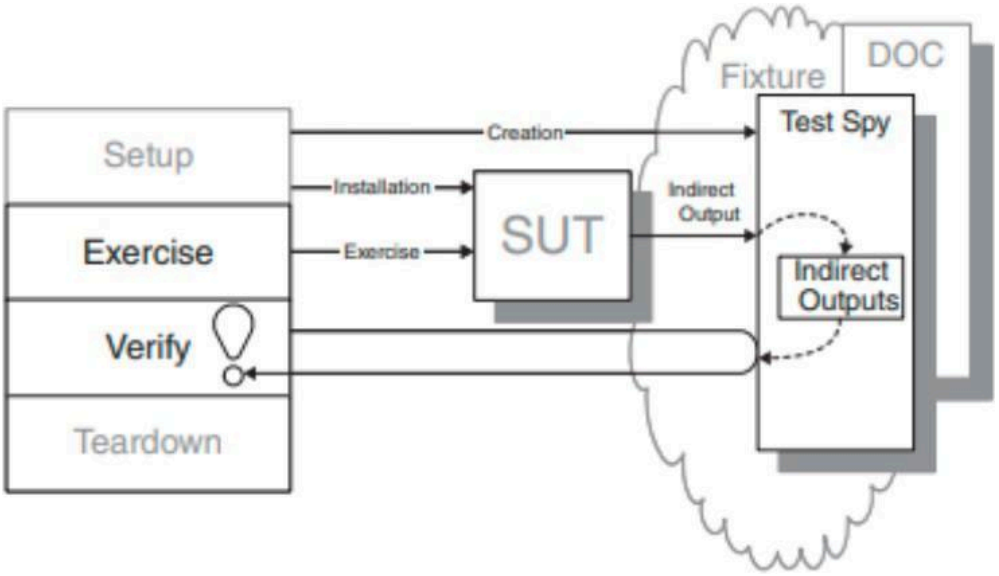



## 2. Test Spy

- Test Stub + registro de outputs
- Es un objeto utilizado en pruebas unitarias que registra información sobre las llamadas realizadas por el código bajo prueba a sus dependencias
- No definen expectativas de llamadas y no generan errores si las llamadas esperadas no ocurren.(Diferencia con Stub)
- Se utilizan para verificar el comportamiento posteriormente, inspeccionando las llamadas registradas.
- útiles para verificar qué llamadas se realizaron y con qué parámetros, permitiendo analizar el comportamiento del código bajo prueba.

**Propósito: registrar outputs indirectos que el SUT envía al DOC.**

- Permite verificar después qué métodos se llamaron y con qué parámetros.
- Similar a un Stub, pero agrega registro.
- Las aserciones se hacen después de que se ejecuta el test.
- Útil cuando se necesita visibilidad sobre lo que hace el SUT, pero sin fallar automáticamente.




**Cuándo debería usarlo**

- Estamos verificando las salidas indirectas del SUT y no podemos predecir los valores de todos los atributos de las interacciones con el SUT con anticipación.
- Nos gustaría tener acceso a todas las llamadas salientes del SUT antes de hacer cualquier afirmación sobre ellas.

## 3. Mock Object

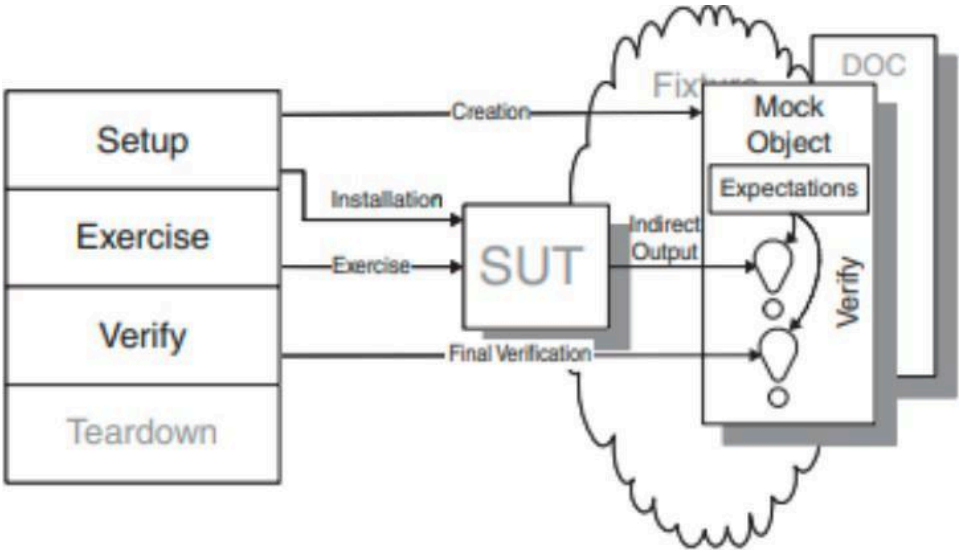
- **Test Stub + verificación de outputs**
- Se configura con expectativas de llamadas específicas
- Se usa para verificar las interacciones entre el código bajo prueba y sus dependencias.
- Se definen expectativas de llamadas y se comprueba si se han cumplido durante la prueba. Si alguna expectativa no se cumple, se genera un error de prueba.
- Reemplazamos un objeto del que depende el SUT con un objeto específico para pruebas que verifica que está siendo utilizado correctamente por el SUT.

**Propósito: verificar interacciones entre el SUT y el DOC.**

- Se configura con **expectativas** (qué métodos deben llamarse, con qué

parámetros, y cuántas veces).
Falla automáticamente si no se cumplen las expectativas.

- Más estricto que el Spy.
- Requiere herramientas o frameworks que permitan definir esas expectativas.




**Cuándo debería usarlo**

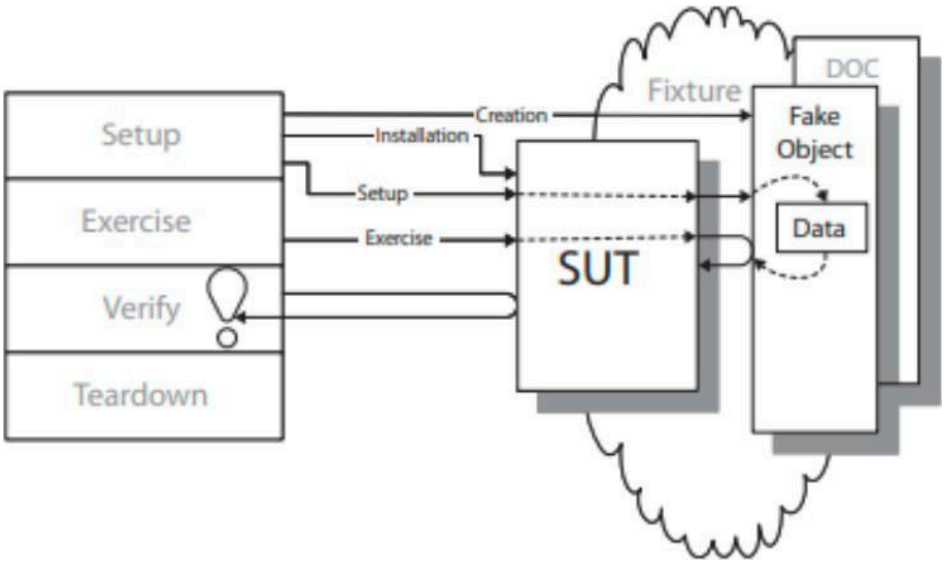
- Para hacer Verificación de Comportamiento cuando no se pueden observar efectos secundarios.
- Si se pueden predecir los argumentos de las llamadas al método.
- No se debe usar si una afirmación fallida no puede reportarse correctamente al Test Runner.

## 4. Fake Object

- **Imitación. Se comporta como el módulo real (protocolos, tiempos de respuesta, etc)**
- Es una implementación simplificada de un objeto real utilizado en pruebas unitarias
- No se configuran para proporcionar respuestas predefinidas o simular interacciones específicas.
- Suelen ser versiones más simples y controladas de las dependencias reales. (diferencia con stubs) Por ejemplo, se pueden crear versiones en memoria de una base de datos o un servicio web para acelerar las pruebas y eliminar la dependencia de componentes externos

**Propósito: reemplazar la implementación real con una más simple o rápida.**

- Tiene lógica real (no solo simulación), pero simplificada.
- Ejemplo clásico: una base de datos en memoria.
- No se usa para verificar llamadas, sino para evitar usar componentes reales lentos, costosos o aún no disponibles.




**Cuándo debería usarlo**

- Siempre que el SUT dependa de otros componentes que no estén disponibles o que dificulten las pruebas y estas requieran secuencias de comportamiento más complejas.

## Test de Aceptación

- Por cada funcionalidad esperada.
- Escritos desde la perspectiva del cliente

# Frameworks

Un framework se puede ver como el resultado de refactorizar una aplicación, no solo para mejorar su calidad interna sino para abstraer lo que es común todas una familia de aplicaciones similares. En ese proceso de refactoring se identifica lo que se mantendrá constante y será, a partir de ese momento, responsabilidad de los desarrolladores del framework y lo que podrá variar de aplicación a aplicación y será responsabilidad de los desarrolladores de aplicaciones que usen el framework.

**Diferencia entre frameworks y librerias:**

**Libreria: *conjunto de "clases útiles"***

- Resuelven problemas comunes a la mayoría de aplicaciones.
- Cada clase de la librería resuelve un problema concreto, es independiente del contexto del uso y generalmente también es independiente de otras clases en la librería
- Nuestro código controla/usa a los objetos de las librerías

**Framework: estructura coherente que define un esqueleto "semi-completo"**

- Aplicación semi-completa y reusable que puede ser especializada para producir aplicaciones a medida
- Conjunto de clases concretas y abstractas, relacionadas (por herencia, composición, envío de mensajes) de manera que resuelven la mayor parte del problema en cuestión (Conforman un todo)
- Los desarrolladores incorporan al framework en sus aplicaciones y lo especializan para cubrir sus necesidades

**IMPORTANTE: En una librería se reusa código mientras que en un framework se reusa una manera de instanciar, de ejecutar y de extender**

## Inversión de control

- Permite que el flujo de control del programa sea invertido. En lugar que de que el código de la aplicación controle el flujo de la ejecución, un

framework o contenedor externo toma el control y llama a los componentes según sea necesario
- Permite al framework determinar qué set de métodos específicos de la aplicación invocar en respuesta a eventos externos
- Entonces
    - Código del framework envia mensajes a mis clases
    - El framework ejecuta mis métodos e instancia mis clases (cosa que normalmente no sucedería)
    - CÓDIGO DEL FRAMEWORK INVOCA A MI CÓDIGO (El código del framework controla al nuestro)

Conclusión: La **inversión de control** es un mecanismo que poseen los frameworks que les permiten ejecutar código hecho por el desarrollador que lo implementa, ante determinados eventos, en lugar que el código de dicho desarrollador invoque al código del framework

Para separar la implementación de la parte constante (a lo que llamamos **frozenspot)** y de las partes que varían (a las que llamamos **hotspots**), se introducen en el framework puntos de extensión. Estos tienen la forma de plantillas que ofrecen ganchos para que, mediante herencia o composición, los programadores de aplicaciones definan el comportamiento variable. Algunos patrones de diseño (particularmente el template method y el strategy) son frecuentemente utilizados para introducir esos puntos de extensión.

La construcción y uso de frameworks es una estrategia para modularizar y reutilizar, que mejora la calidad del software y aumenta la productividad de quienes lo hacen. Un framework, visto como módulo de software, no solo encapsula operaciones y estructuras de datos reusables sino que las combina para resolver, con demostrada eficacia, los requerimientos comunes a una familia de aplicaciones. En pocas palabras, encapsula el diseño y la implementación de la parte más compleja de dichas aplicaciones y permite su reutilización.

Un framework es en sí mismo un módulo de software reutilizable. Muchas aplicaciones se construirán sobre él. El framework será independiente de las aplicaciones. Sin embargo, las aplicaciones dependerán de que la interface (conjunto de puntos de extensión) y comportamiento del framework se mantenga estable. Esto es importante dado que, a lo largo de su vida, el framework será foco de contínuo refactoring y de

mejoras para eliminar bugs y hacerlo más efectivo.

## FrozenSpot

- Elementos del framework que no se pueden modificar (marca indeleble)
- Conjunto de características que comparten todas las aplicaciones creadas por el mismo framework

## HotSpot

- Puntos de extensión que ofrece el framework que nos permiten introducir variantes y así construir aplicaciones diferentes
- Los puntos de extensión pueden implementarse en base a herencia o en base a composición

## Plantillas y ganchos

Las plantillas y los ganchos son una estrategia de programación comúnmente utilizada para introducir puntos de variabilidad (hotspots) en los frameworks orientados a objetos. Pueden implementarse utilizando herencia o composición como mecanismos para separar los aspectos comunes de los aspectos variables.

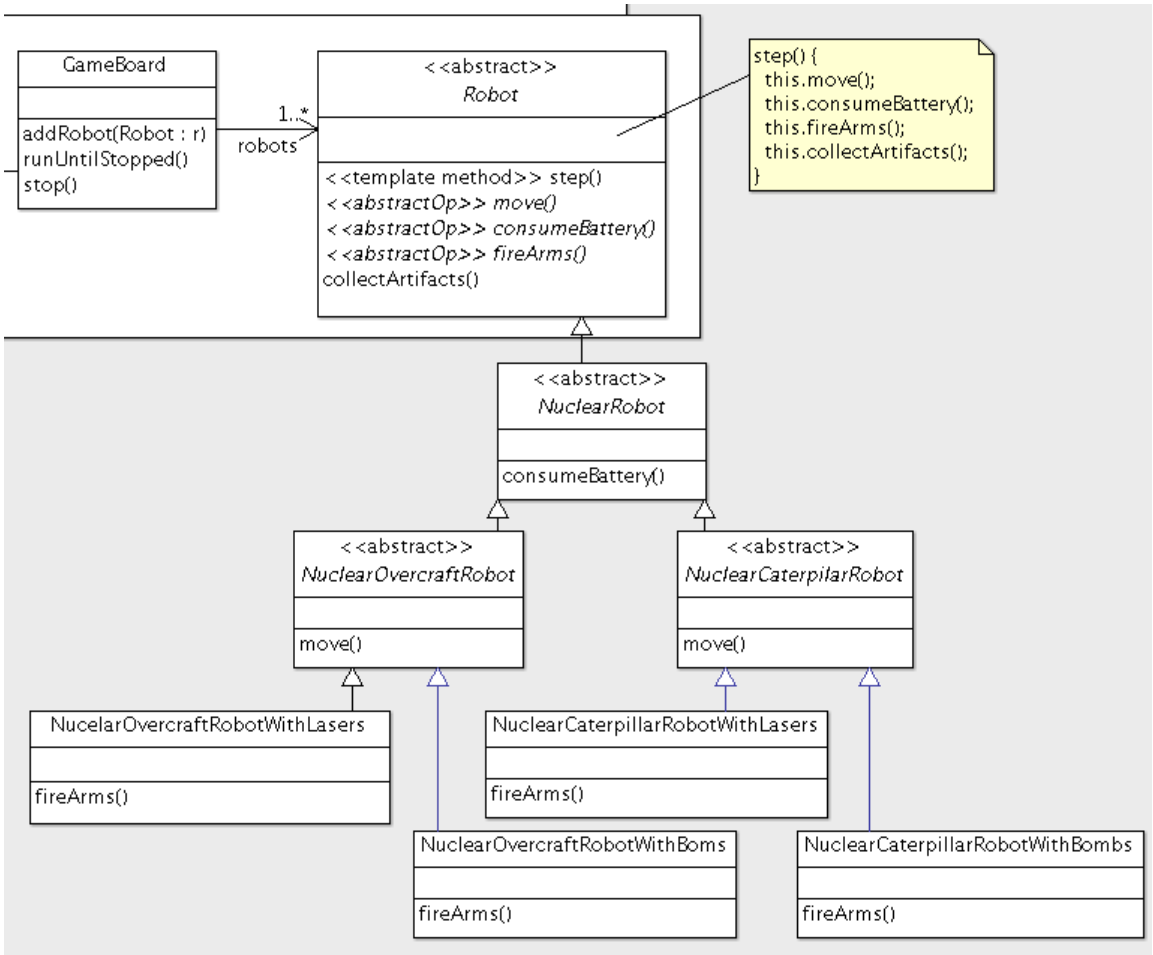
## Plantillas y herencia:

El método plantilla es parte del frozenspot mientras que las operaciones abstractas (o los ganchos) dan lugar a la implementación de los hotspots.

La explosión en el número de subclases y la duplicación de código evidencian las limitaciones de la herencia como estrategia para separar plantillas y ganchos. En situaciones como estas puede explorarse la composición como una estrategia alternativa.

**Si uso herencia, la plantilla se implementa en una clase abstracta y los ganchos en sus subclases .**

Ejemplo de la utilización de **herencia** para introducir puntos de variabilidad:




## <u>Plantillas y composición</u>:

No solo lo separa en métodos diferentes, sino que también lo encapsula en objetos diferentes

Importante:

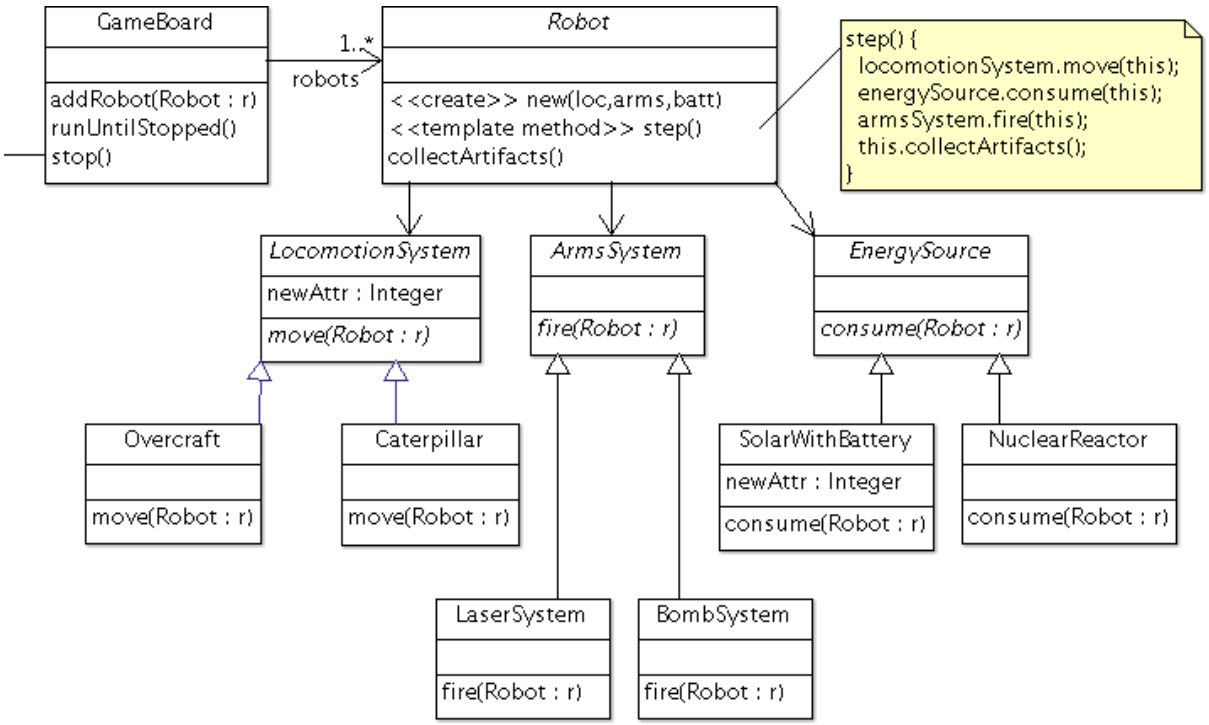
Ahora, cuando un robot recibe el mensaje *step(),* sigue utilizando su método plantilla (que garantiza el cumplimiento de los pasos canónicos como lo marca la especificación del framework). Sin embargo el método plantilla ya no envía mensajes al objeto mismo (como lo hacía en el caso de herencia) sino que delega las operaciones en sus partes. Entonces, <u>la plantilla está en el robot, y los ganchos en las partes.</u>

Cuando el robot delega las operaciones gancho en sus partes, <u>se pasa el mismo como parámetro</u>. Esto es necesario porque las partes ya no tienen

acceso a las variables de instancia del robot ni de las otras partes. En consecuencia al implementar los métodos ganchos (en las partes) sólo se puede utilizar los métodos que el robot haga públicos. Esto resulta en un escenario más restrictivo que el que utilizaba herencia.

**Si uso composición, un objeto implementa la plantilla y delega la implementación de los ganchos en sus partes.**

Ejemplo de la utilización de **composición** para introducir puntos de variabilidad:




En resumen:

- Las plantillas y los ganchos se pueden utilizar en un framework para separar lo que se mantiene constante (frozenspot) de lo que varía (hotspots).
- Las plantillas implementan lo que se mantiene constante, los ganchos lo que varía.
- Se puede utilizar herencia o composición para separar la plantilla de los ganchos.
- Si uso herencia, la plantilla se implementa en una clase abstracta y los ganchos en sus subclases (Template Method).
- Si uso composición, un objeto implementa la plantilla y delega la implementación de los ganchos en sus partes.

- Con herencia:
    - es más simple para casos con pocas alternativas y/o pocas combinaciones
    - al implementar los métodos gancho puedo utilizar las variables de instancia y todo el comportamiento heredado de la clase abstracta
    - si hay muchas variantes o combinaciones, empiezo a tener muchas clases y duplicación de código

- Con composición:
    - evita la duplicación de código y el creciente número de clases cuando existen muchas alternativas y combinaciones posibles.
    - al implementar los métodos gancho debo pasar como parámetro todo lo que necesiten - no tienen acceso a las variables de instancia del objeto.
    - puedo cambiar el comportamiento en tiempo de ejecución sin mayor dificultad.

## Frameworks de caja blanca

- Utilizan herencia en sus puntos de extensión
- Debemos conocer el framework para poder usarlo(como se concetan las piezas, como interactuan.
- Loop de control suele ser Template method

## Frameworks de caja negra

- Utilizan composición en sus puntos de extensión
- No hace falta conocer el funcionamiento grandes detalles internos del framework
- Loop de control suele ser manejado por un contenedor o configurador

**¿Por qué se dice que en las frameworks white box no hay encapsulamiento?**

- Porque EchoServer hereda el loop de control no existe encapsulamiento

⇒ es una Caja Blanca

- La instanciación hereda y completa el loop de control
- Es posible que requiera agregar métodos a clases del framework
- Demanda conocimiento del código del framework

## CONVERTIR FROZENSPOTS EN HOTSPOTS

```
72                  if (inputLine.equalsIgnoreCase("")) {
73                      break; // Client requested to close the connection
74                  }
```

Ejercicio: como podría convertir ese FrozenSpot en un HotSpot?

1. Extraigo la lógica en un método configurable (hook method):

```
protected boolean connectionClosure(String inputLine) {
    return inputLine.equalsIgnoreCase("");
}
```

2. Reemplazo por la llamada al método hook en el loop de conexión:

```
if (this.connectionClosure(inputLine)) {
    break;
}
```

3. En la clase EchoServer (instanciación)

```
@Override
protected boolean connectionClosure(String inputLine) {
    return inputLine.equalsIgnoreCase("xd");
}
```

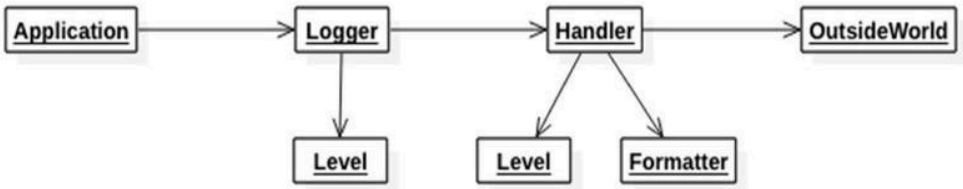
**Para convertir ese FrozenSpot en un HotSpot, hay que extraer la lógica fija (como inputLine.equals("")) en un método que pueda ser sobrescrito por las instanciaciones del framework.**

### JAVA UTIL LOGGIN:

- Agregamos código de loggin a nuestra aplicación para entender lo que pasa con ella
- Los logs son útiles para desarrolladores, administradores y usuarios

## Arquitectura visible

- Logger: objeto al que le pedímos que emita un mensaje de log
- Handler: encargado de enviar el mensaje a donde corresponda
- Level: indica la importancia de un mensaje y es lo que mira un
- Logger y un Handler para ver si le interesa
- Formater: determina cómo se "presentará" el mensaje

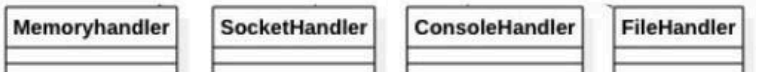



```
//Obtener instancias del logger y almacenarlo en una variable:
Logger.getLogger("app");
appLog= Logger.getLogger("app");
//Setear el nivel del logger:
Logger.getLogger("app").setLevel(Level.OFF);
appLog.setLevel(Level.SEVERE);
Logger customersLogger = Logger.getLogger("ecommerce.customers");

//Establece que el nivel mínimo de mensajes que este logger aceptará es
```

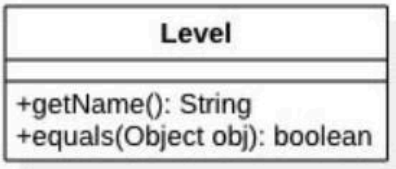
```
WARNING (solo logueará mensajes de nivel WARNING, SEVERE, o superior)
customersLogger.setLevel(Level.WARNING);

/* Crea un manejador de logs que guarda los mensajes en un archivo
llamado ecommerce-customers.log
Un Handler es lo que se encarga de dónde se envía el log (consola, archivo,
etc) */
FileHandler customersLoggerhandler= new
FileHandler ("ecommerce-customers.log");

//Define como se formatea el mensaje de log
customersLoggerHandler.setFormatter(new SimpleFormatter());

/* Asocia el FileHandler al logger (le indica que escriba los mensajes al
archivo)
   Un logger puede tener multiples Handlers al mismo tiempo */
customersLogger.addHandler(customersLoggerHandler);
```

# Handler

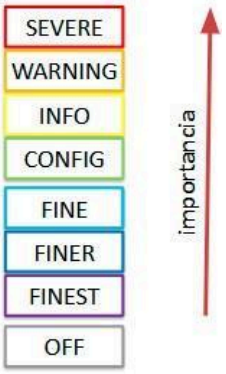
- Recibe los mensajes del Logger y determina como "exportarlos"
- Instancias de MemoryHandler, ConsoleHandler, FileHandler, o SocketHandler
- Puede filtrar por nivel
- Tiene un Formatter




# Level

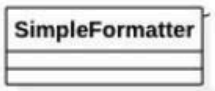


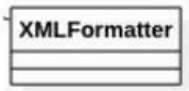

- Representa la importancia de un mensaje
- Cada vez que pido que se loggee algo, debo indicar un nivel
- Los Loggers y Handler comparan el nivel de un cada mensaje con el suyo para decidir si les interesa o no
- Si te interesa un nivel, también te interesan los que son más importantes que ese
- Hay niveles predefinidos, en variables estáticas de la clase Level (p.e., Level.OFF)



# Formatter

- El Formatter recibe un mesaje de log (un objeto) y lo transforma a texto
- Son instancias de: SimpleFormatter o XMLFormatter
- Cada handler tiene su formatter
  - Los FileHandler tienen un XMLFormatter por defecto
  - Los ConsoleHandler tienen un SimpleFormatter por defecto



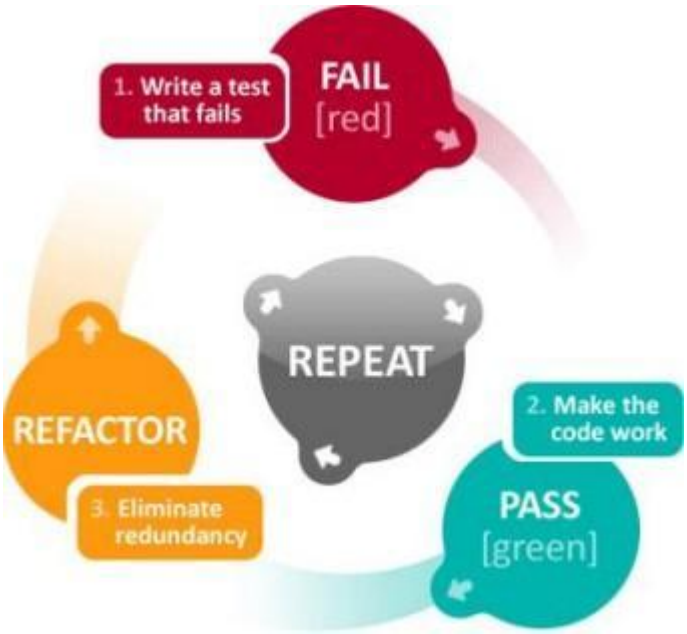



# TDD

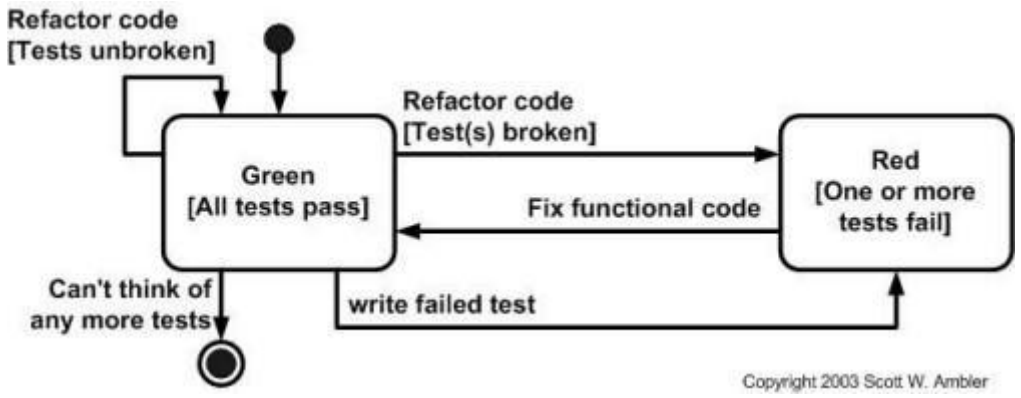
- **TDD = eXtreme(eXtreme(Programming))**
- **Foco en el desarrollo (osea: el programador)**

## Ciclo de TDD

1. Red: Write a failing test for a new feature or requirement.
2. Green: Write just enough code to make the failing test pass.
3. Refactor: Improve the code while ensuring the tests still pass.
4. (Goto 1)Repeat: Continue with another new test case, repeating the cycle.




- ➔ Se escriben tests funcionales para capturar use cases que se validan automáticamente
- ➔ Se escriben tests de unidad para enfocarse en pequeñas partes a la vez y aislar los errores
- ➔ No agregar funcionalidad hasta que no haya un test que no pasa porque esa funcionalidad no existe.
- ➔ Una vez escrito el test, se codifica lo necesario para que todo el test pase.
- ➔ Pequeños pasos: un test, un poco de código
- ➔ Una vez que los tests pasan, se refactoriza para asegurar que se mantenga una buena calidad en el código




## Proceso de TDD

- ➔ Captura de Requerimientos
  - ◆ Historias cortas
  - ◆ Menor distancia entre Cliente y Programador
- ➔ Herramientas de testing (xUnit)
  - ◆ Testing Unidad
  - ◆ Test Regresión
  - ◆ Testing Integración (criterio de aceptación)
- ➔ Integración Continua

## Beneficios

- Historias de Usuario y Tareas más Pequeñas.
- Criterios de Aceptación Claros y Realizables
- Priorización Basada en el Valor de Negocio y el Riesgo
- Diseño Emergente
- Refinamiento Continuo
- Colaboración y Entendimiento Compartido

## Requerimientos

- Comunicación y Colaboración Efectivas (que incluye al “cliente”)
- Backlog Refinado con Criterios de Aceptación Testeables
- Equipo con Cultura orientada al Testing
- Frameworks de Pruebas Unitarias Adecuados
- Herramientas de Cobertura de Código (Opcional Inicialmente)
- Integración Continua (CI)

## Reglas:

- ➔ Diseñar incrementalmente:
  - ◆ teniendo código que funciona como feedback para ayudar en las decisiones entre iteraciones.
- ➔ Los programadores escriben sus propios tests:
  - ◆ no es efectivo tener que esperar a otro que los escriba por ellos.
- ➔ El diseño debe consistir de componentes altamente cohesivos y desacoplados entre si:

  mejora evolución y mantenimiento del sistema.

  Hay herramientas que automatizan TDD.

- **Test-Driven Development (TDD)** es una práctica de diseño, no solo de testing.
- Se basa en **especificar el comportamiento deseado antes de implementar**.
- El foco está en la calidad, simplicidad y evolución del diseño